

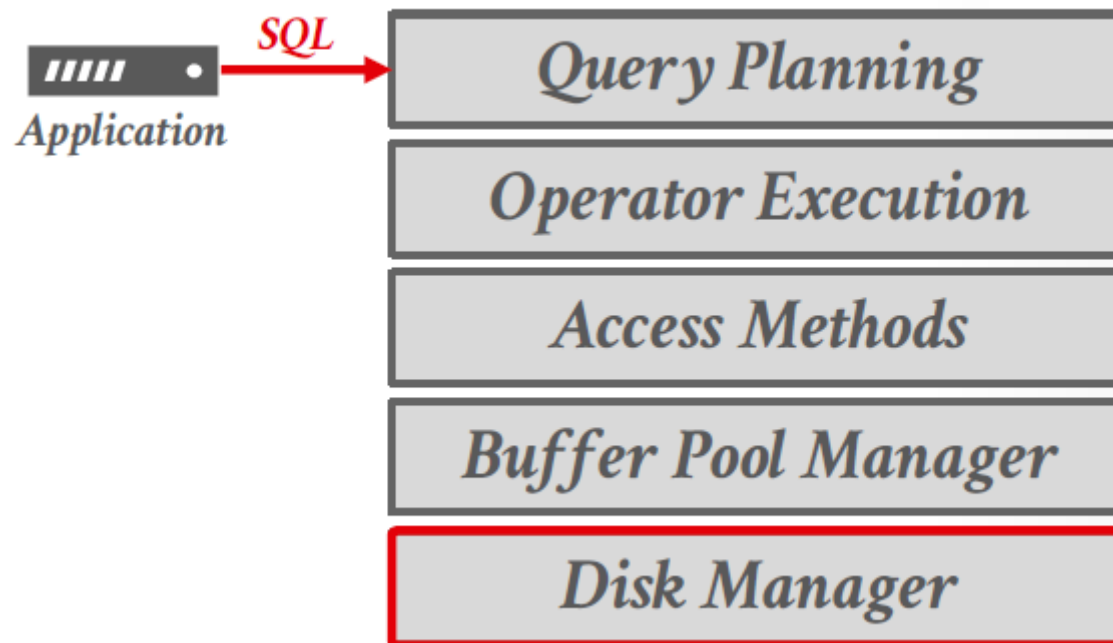
Curso _____
Estructuras de datos

**UNIVERSIDAD
DE ANTIOQUIA**

Clase 4 – Archivos (Parte 1)

Sobre el curso

- En este curso se abordaran algunos conceptos relacionados con el diseño y la implementación de sistemas de gestión de bases de datos orientados a disco.
- No es un curso sobre cómo usar una base de datos para construir aplicaciones ni sobre cómo administrar una base de datos.

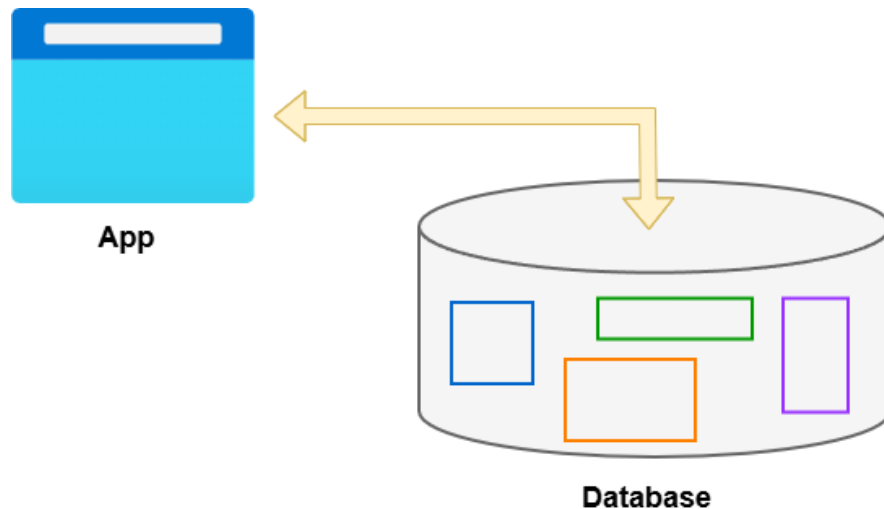


Base de datos

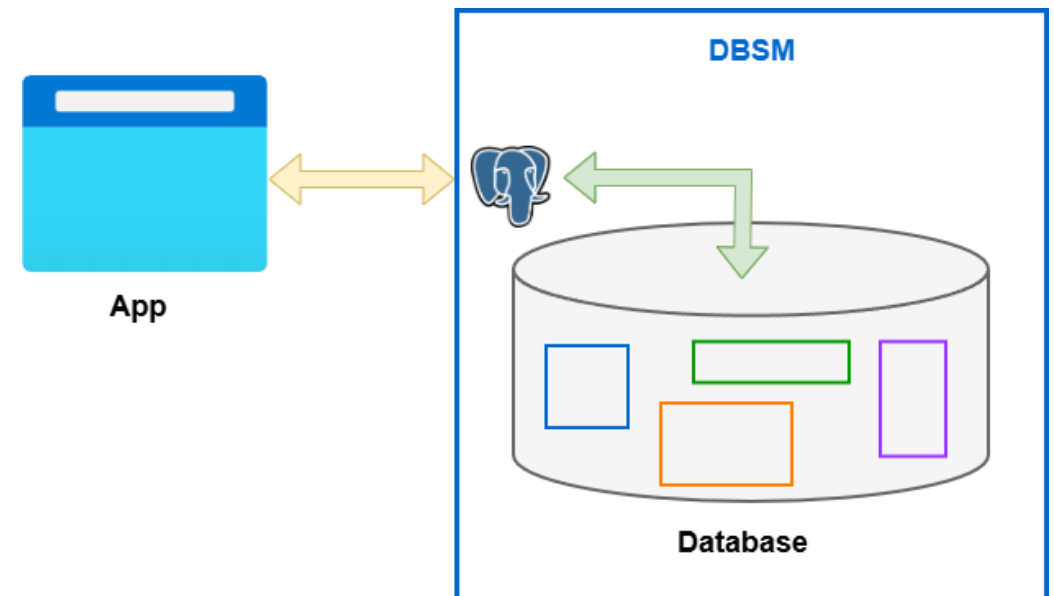
La clase anterior tratamos algunos de los siguientes conceptos

- Base de datos
- Manejo de bases de datos:
 - **Forma antigua:** Sistemas basados en archivos
 - **Actual:** Motor de bases de datos (DBSM)

Sistemas basados en archivos



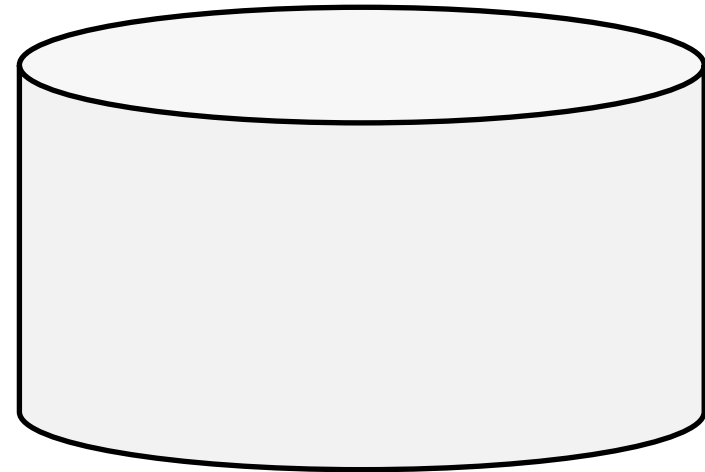
Motor de bases de datos



Representando los datos

¿Cómo un DBMS representa la base de datos en archivos en el disco?

ID	Last Name	First Name	Age	Salary
123	Simpson	Homer	31	\$400
443	Simpson	Marge	32	\$140
244	Flanders	Ned	55	\$300
134	Skinner	Seymour	55	\$400

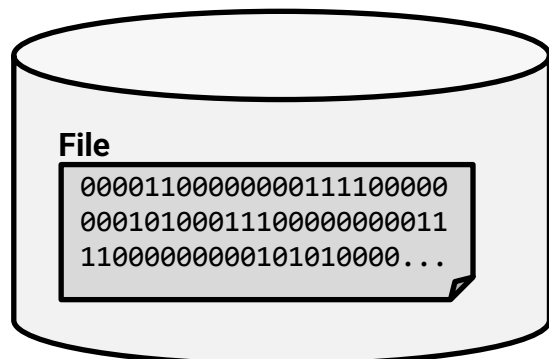


Representación de los datos

Sistema operativo

- Un archivo es simplemente una secuencia de bytes o bloques de disco.
- No importa lo que representan los bytes.
- El sistema operativo administra la memoria decidiendo que archivos mantener en RAM.

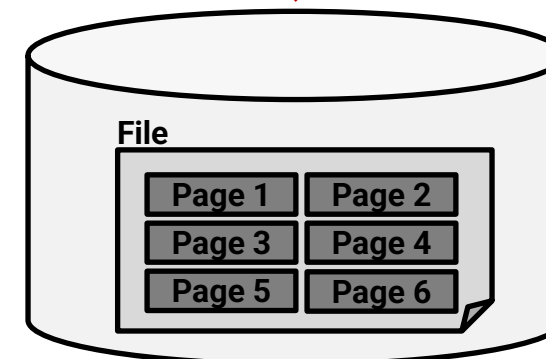
ID	Last Name	First Name	Age	Salary
123	Simpson	Homer	31	\$400
443	Simpson	Marge	32	\$140
244	Flanders	Ned	55	\$300
134	Skinner	Seymour	55	\$400



DBMS

- El DBMS va mas allá de un simple archivo
- Los datos tienen significado.
- El DBMS tiene su propio **Buffer Pool** para decidir que paginas mantener en memoria

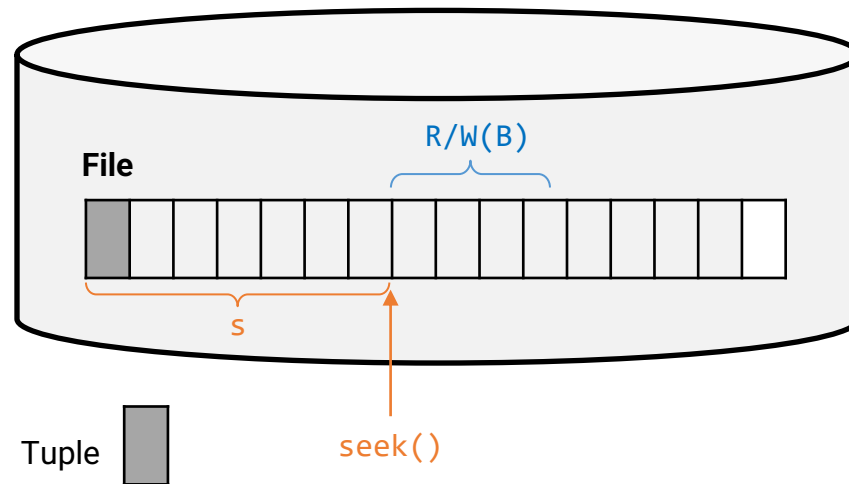
ID	Last Name	First Name	Age	Salary
123	Simpson	Homer	31	\$400
443	Simpson	Marge	32	\$140
244	Flanders	Ned	55	\$300
134	Skinner	Seymour	55	\$400



Representación de los datos

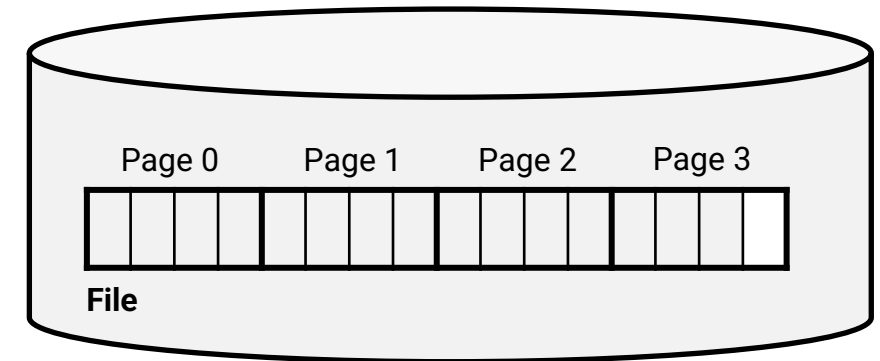
Sistema operativo

- El SO proporciona un API que permite:
 - El posicionamiento es por bytes (Saltar s bytes)
 - Lectura/Escritura por bytes (B bytes)



DBMS

- El DBMS trabaja por paginas.
 - Favorece el acceso secuencial (mas rápido).
 - El salto se hace a través de las paginas.
 - Lectura/Escritura por paginas.



Archivos, paginas, registros

- **Bases de datos (Parte Lógica)**

- **Registro (fila):** Unidad básica de datos en bases de datos relacionales.
- **Relación (tabla):** Grupo de registros.

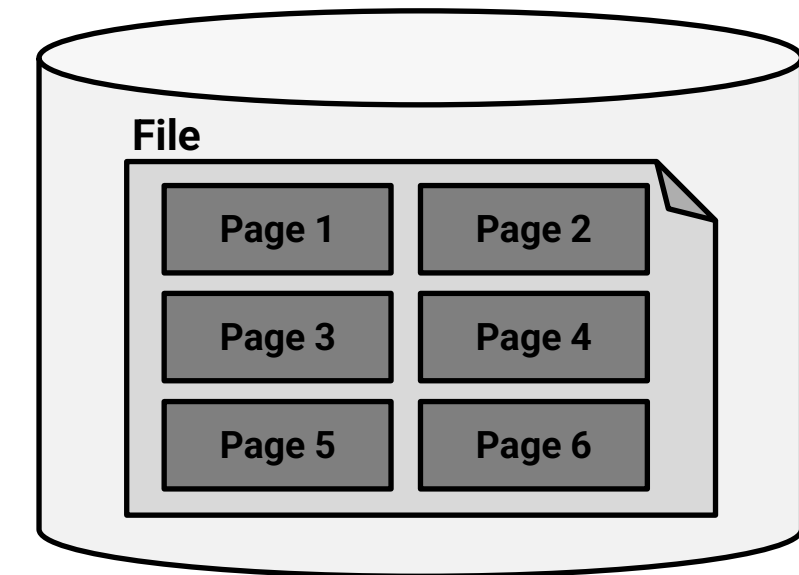
- **Disco (Parte física)**

- **Pagina:** Es la unidad básica de datos para el disco.
- **Transferencia:** Representa la unidad mínima de movimiento entre el disco y la memoria (I/O).

Table

ID	Last Name	First Name	Age	Salary
123	Simpson	Homer	31	\$400
443	Simpson	Marge	32	\$140
244	Flanders	Ned	55	\$300
134	Skinner	Seymour	55	\$400

Record



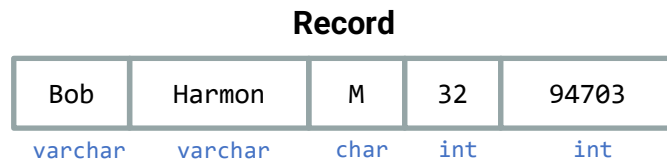
Representaciones

Preguntas claves

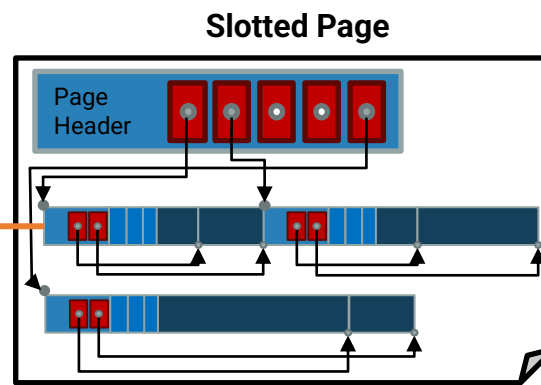
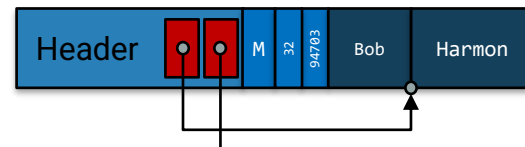
- ¿Como organizar las paginas (pages) dentro de un archivo?
- ¿Cómo organizar los registros (records) dentro de una pagina (page)?

Relation (table)

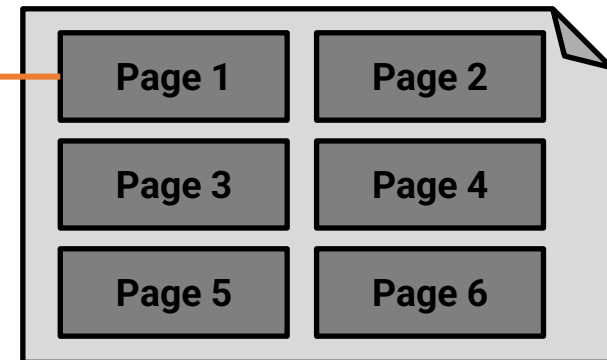
ID	Last Name	First Name	Age	Salary
123	Simpson	Homer	31	\$400
443	Simpson	Marge	32	\$140
244	Flanders	Ned	55	\$300
134	Skinner	Seymour	55	\$400



Byte Representation of Record



File



Representaciones

Una pausa antes de seguir – Observación de claridad importante

- Los datos se almacenan en orden lineal
 - 1 byte por posición
 - Las direcciones de memoria están ordenadas.
 - Las direcciones de disco están ordenadas.

0	1	2	3	4	6	7	8	9	10	11	12	13	14	15	16	...		
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	-----	--	--

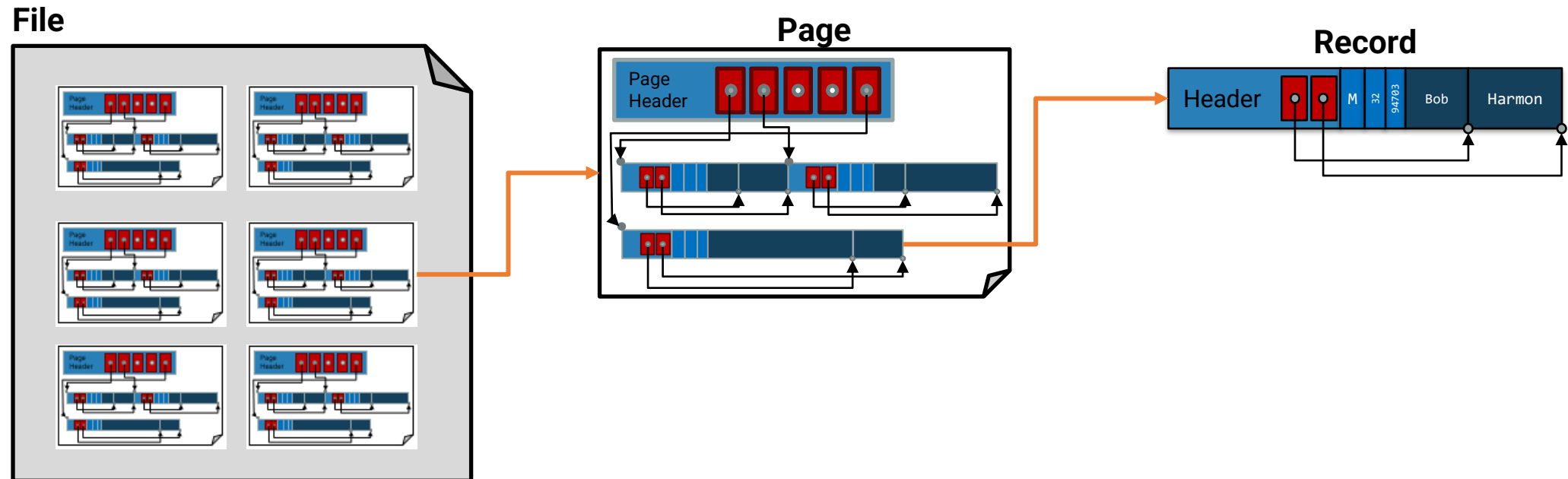
- Para facilitar la representación, se va a emplear una representación donde la representación lineal será mostrada en un esquema rectangular.

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	16
16	...		

Archivos de paginas de registros

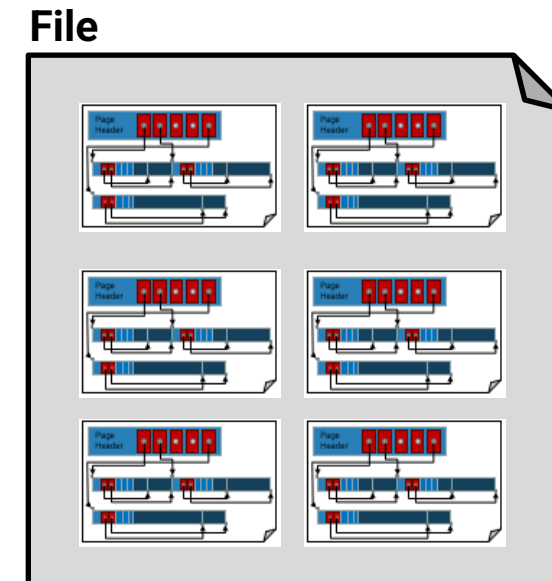
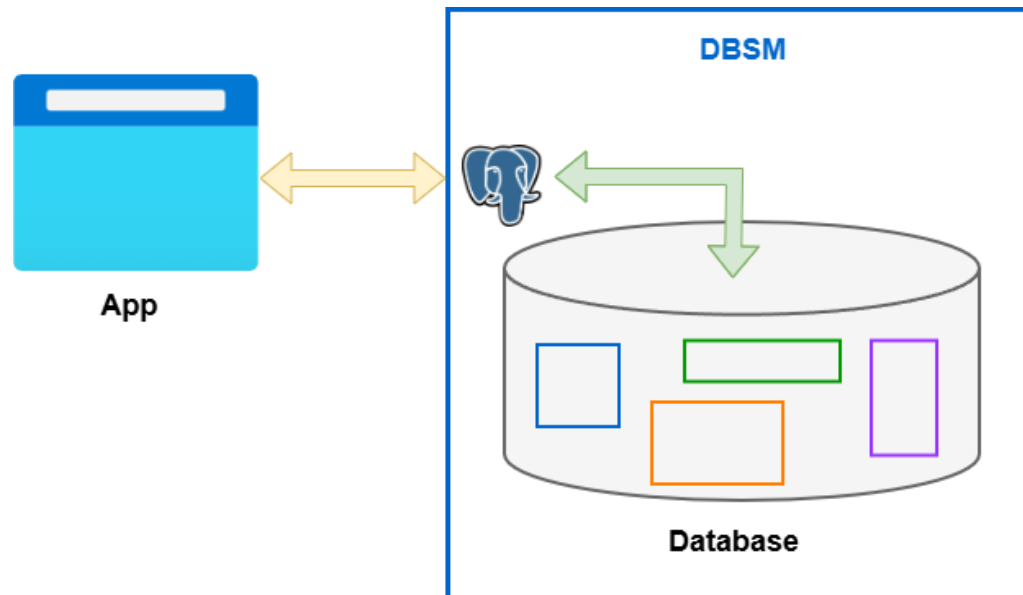
Overview

- Las tablas se almacenan como archivos lógicos (**Files**)
- Los archivos Consisten en páginas (**pages**)
- Las páginas contienen una colección de registros (**records**)



Preguntas claves

- Si el usuario ve tablas con filas y columnas, ¿por qué el disco duro solo ve un rompecabezas de páginas de 8KB?
- ¿Cómo realiza el DBMS la gestión de memoria?
 - ¿Cómo sabe el motor exactamente en qué lugar específico del disco está el dato sin tener que leer archivos de terabytes de principio a fin?
 - ¿Cómo es posible modificar datos de forma instantánea si el disco duro es el componente más lento de toda la computadora?



Buffer Pool (Buffer de la base de datos)

Un **buffer** es una parte de la memoria física utilizada para almacenar datos intermedios entre el disco y los procesos.

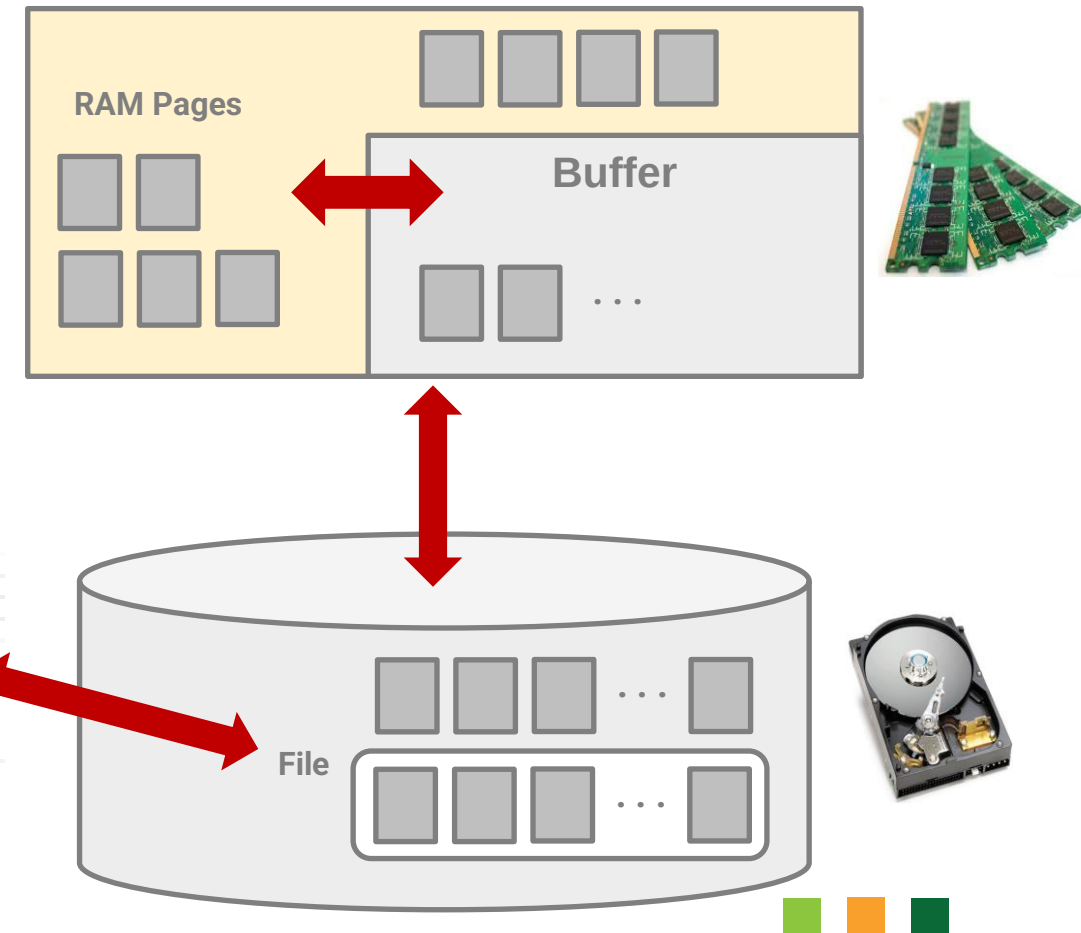
- La base de datos mantiene su propio buffer (¿Por qué? ¿Acaso el sistema operativo no hace ya esto?)
 - La base de datos conoce mejor los patrones de acceso.
 - La recuperación y el logging requieren la capacidad de forzar escrituras a disco.
 - El administrador de buffers (Buffer Manager) maneja las políticas de reemplazo de páginas.

Las bases de datos típicamente configuran el tamaño de página en RAM igual al tamaño del bloque de E/S (por ejemplo, 64 MB).

⇒ Para nuestro caso, **Página**, **Bloque de E/S** y **Bloque de Disco** para referirnos a lo mismo.

Table

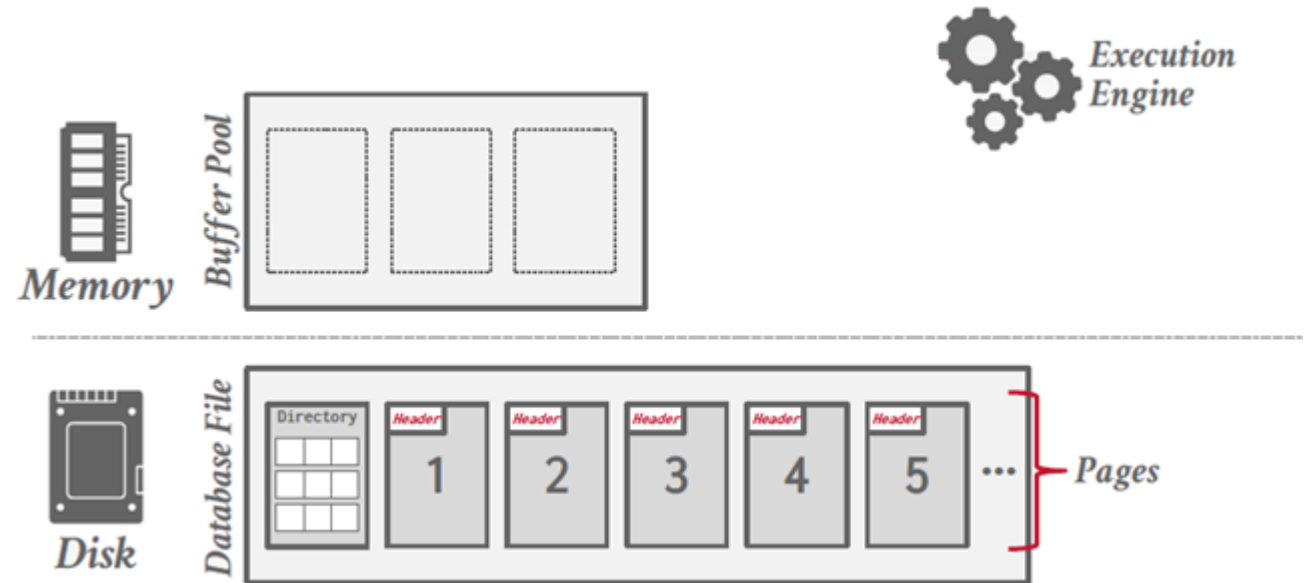
Nombre	Rela	Blqs	Reserva
tab1	Cont	100 000 000	
tab2	Cont	100 000 000	
tab3	Cont	100 000 000	
tab4	Cont	100 000 000	
tab5	Cont	100 000 000	
tab6	Cont	100 000 000	
tab7	Cont	100 000 000	
tab8	Cont	100 000 000	
tab9	Cont	100 000 000	
tab10	Cont	100 000 000	



Gestión de paginas en el Buffer Pool

Para la gestión del buffer (de manera propia) el DBMS divide el trabajo para optimizar el patrones de acceso a los datos:

- **Administrador de Espacio en Disco:** Encargado de la persistencia física.
- **Buffer Manager:** Garantiza que los niveles superiores operen exclusivamente en la RAM.



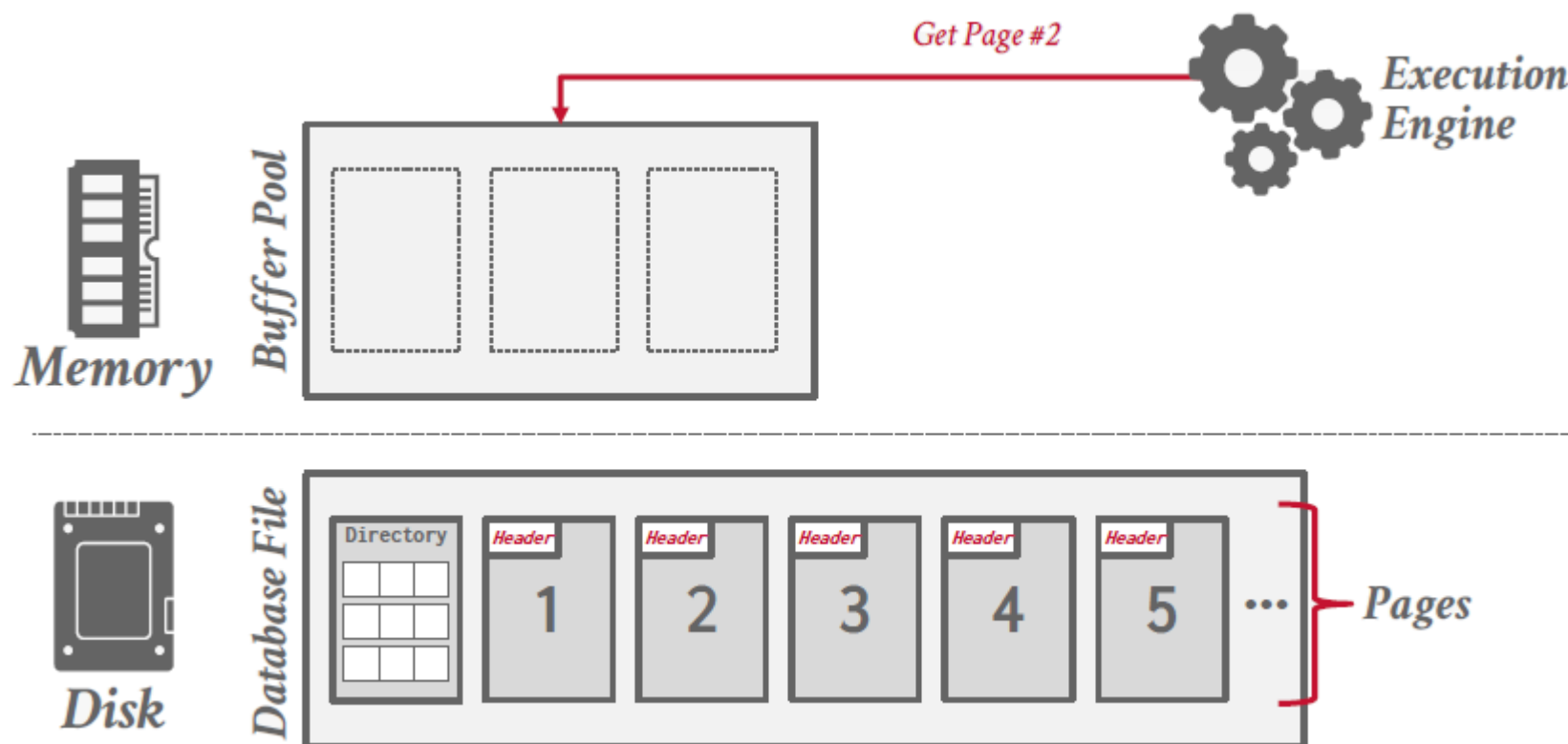
Si solo quiero cambiar una palabra en una fila que pesa 50 bytes, ¿por qué el motor de la base de datos se toma la molestia de cargar 8192 bytes (8KB) completos a la memoria?

Respuesta:

- Recordemos el concepto de jerarquía de memoria.
- Vamos a analizar el procedimiento por pasos.

Gestión de paginas en el Buffer Pool

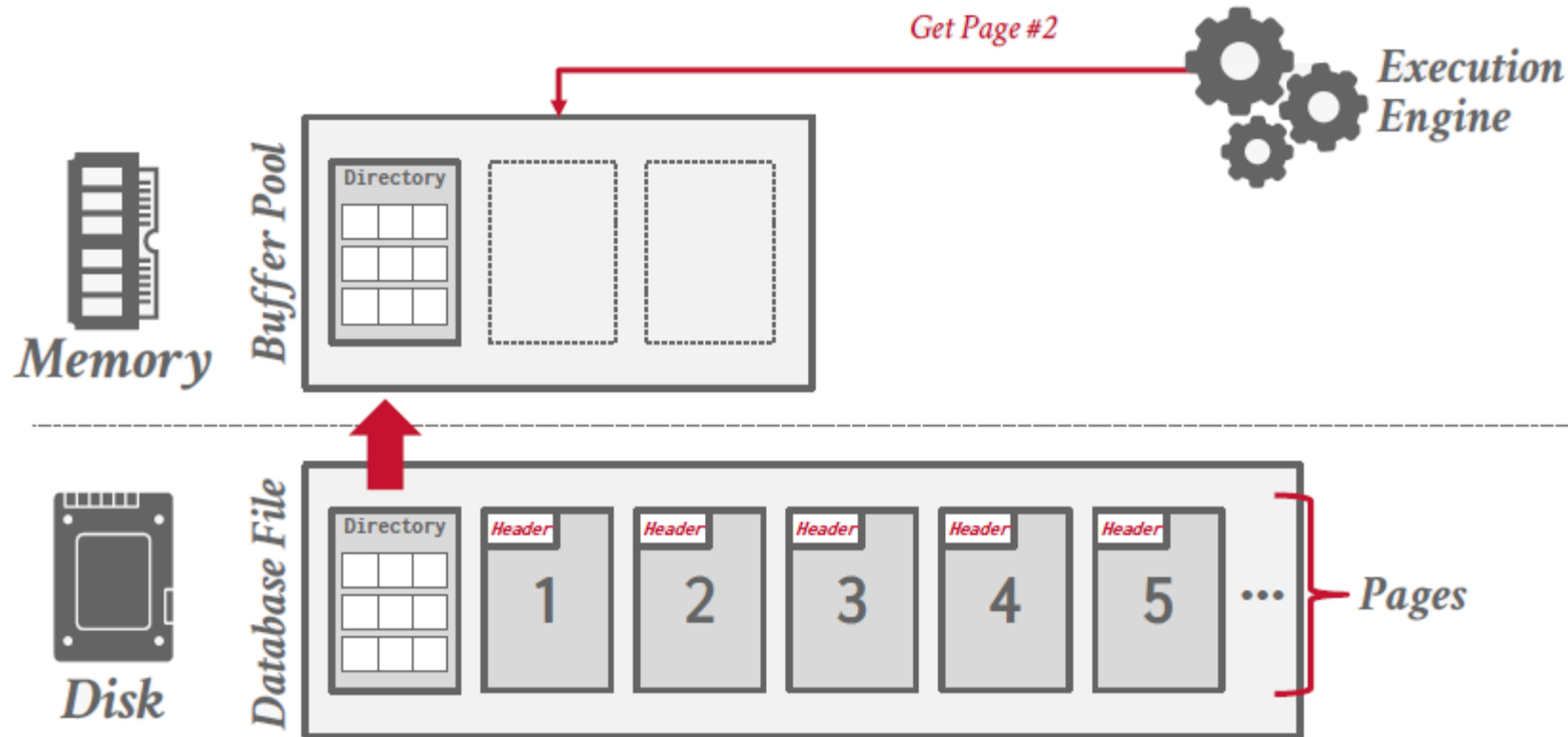
Paso 1 - Solicitud de Datos por el Motor de Ejecución



El proceso comienza cuando el **Motor de Ejecución (Execution Engine)** requiere acceso a una página específica (ej. **Page #2**) para procesar una consulta. La solicitud se envía directamente al **Buffer Pool**

Gestión de paginas en el Buffer Pool

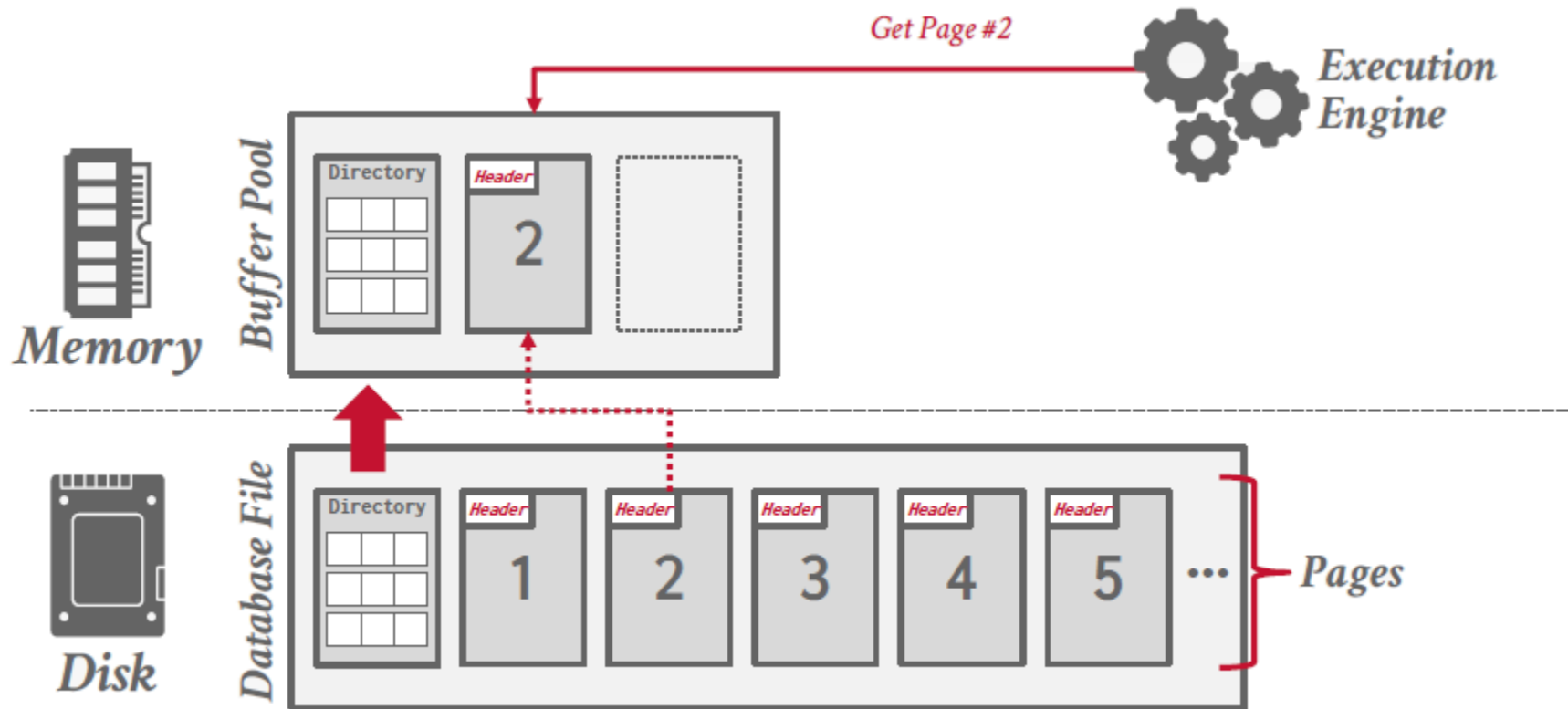
Paso 2 - Verificación del Directorio de paginas y Detección de **Buffer Miss**



El Buffer Manager consulta su **Directorio de Páginas** interno para verificar si la página solicitada ya reside en la memoria RAM. En este caso, ocurre un **Buffer Miss** (la página no está cargada y debe buscarse en el almacenamiento físico).

Gestión de paginas en el Buffer Pool

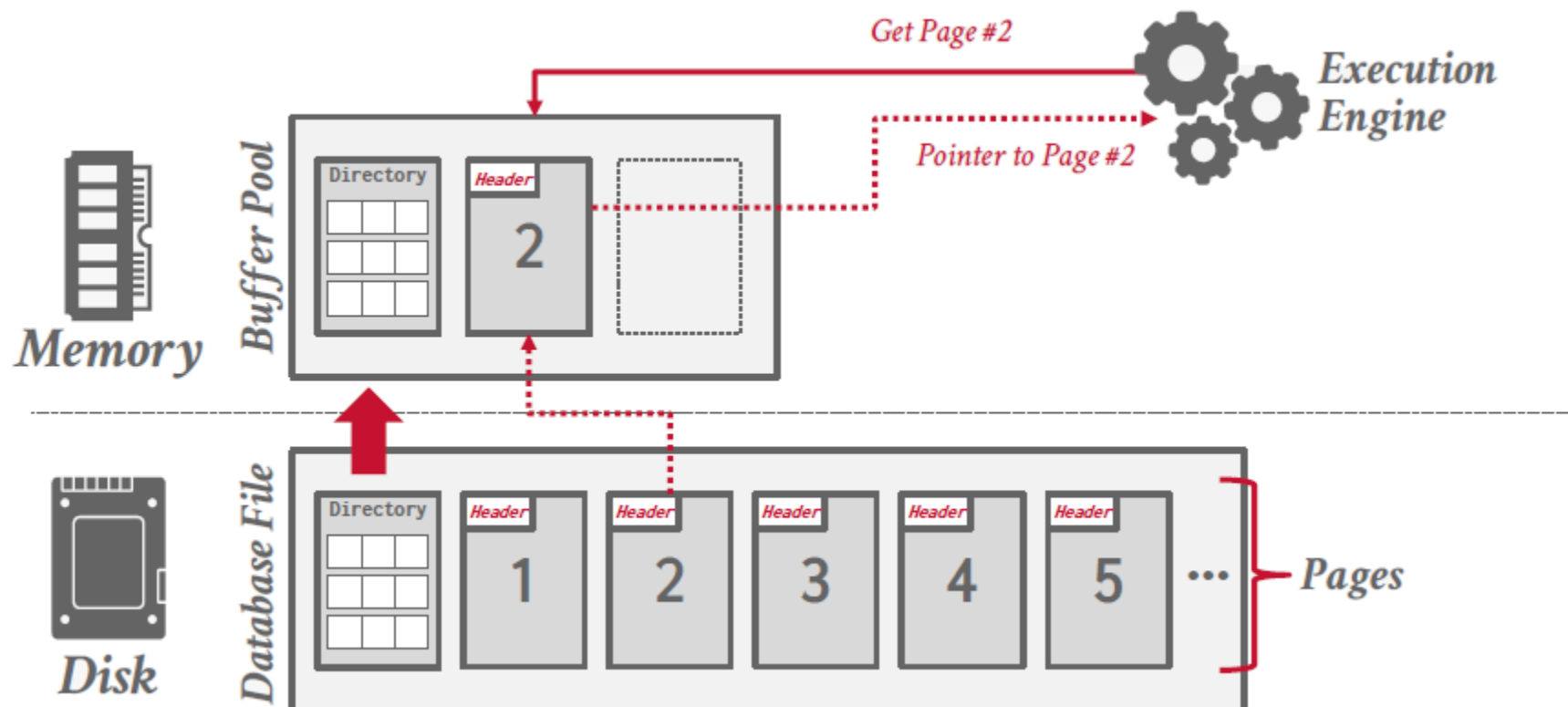
Paso 3 - Operación de E/S: Recuperación del Bloque Físico



El Buffer Manager solicita al Administrador de Disco que localice la **Page #2** en el archivo de la base de datos. La página se lee del disco y se copia en un marco (**frame**) vacío dentro del Buffer Pool en la memoria principal.

Gestión de paginas en el Buffer Pool

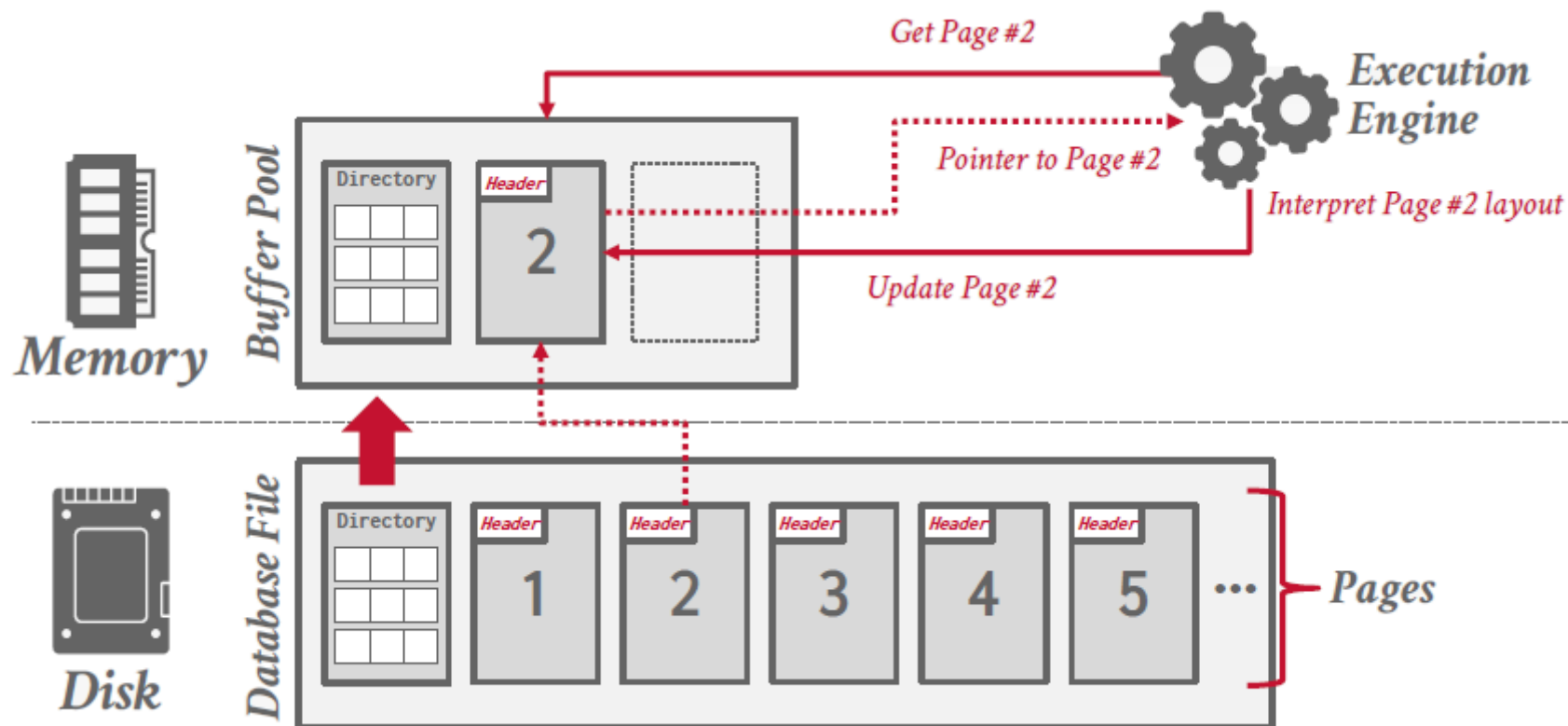
Paso 4 - Operación de E/S: Recuperación del Bloque Físico



Una vez que la página está disponible en RAM, el Buffer Manager devuelve un puntero (dirección de memoria) al Motor de Ejecución. De esta forma, el motor sabe exactamente dónde trabajar sin tener que conocer detalles del archivo físico.

Gestión de paginas en el Buffer Pool

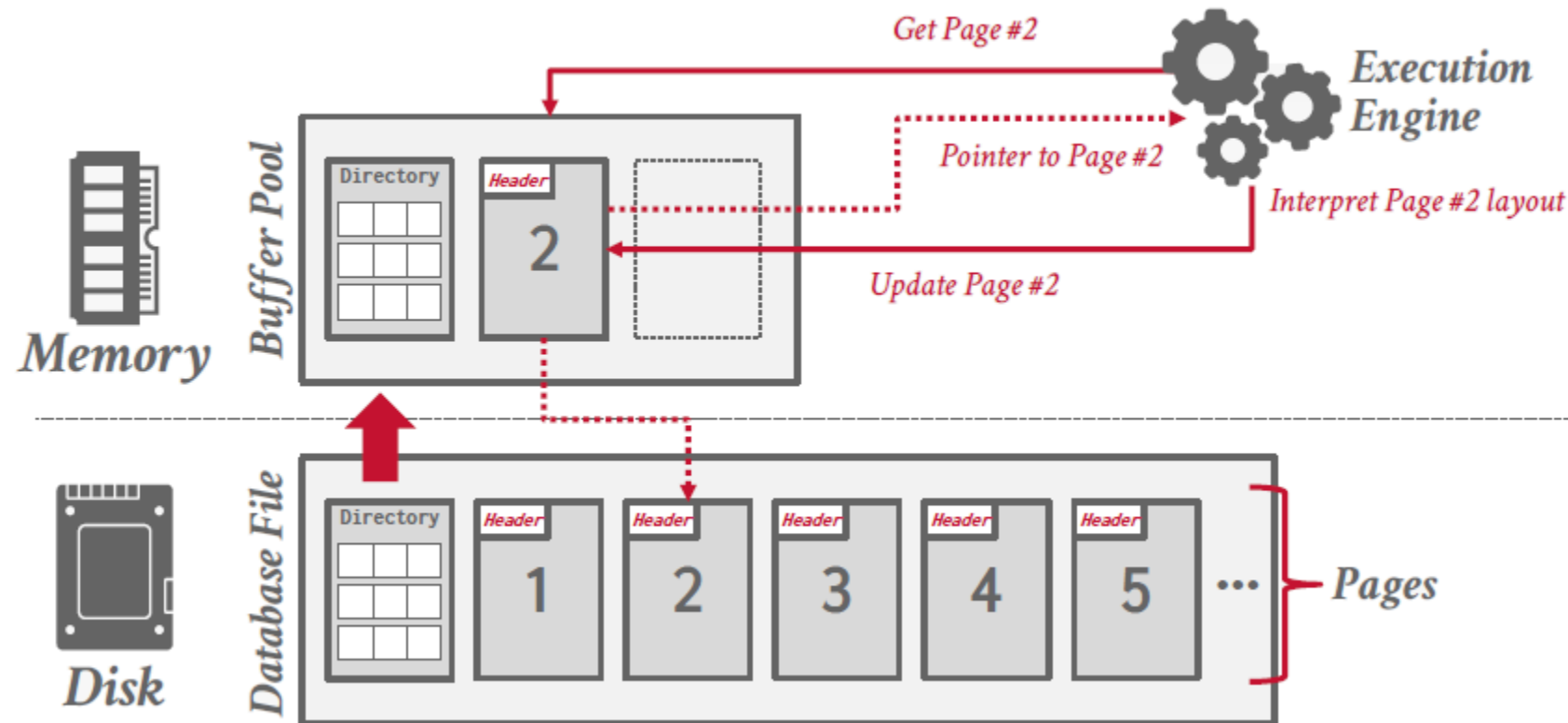
Paso 5 - Vinculación Lógica mediante Punteros de Memoria



El Motor de Ejecución interpreta el formato de la página y realiza las modificaciones necesarias (**update**). Es vital notar que este cambio ocurre únicamente en la memoria RAM en este momento, lo que garantiza una respuesta inmediata.

Gestión de paginas en el Buffer Pool

Paso 6 - Actualización de Registros y sincronización



Tras la modificación, la **Page #2** permanece en el Buffer Pool marcada como '**Dirty Page**' (página sucia). Esta no se escribirá de vuelta al disco de inmediato, sino que el Buffer Manager esperará un momento óptimo para sincronizarla con el archivo físico.

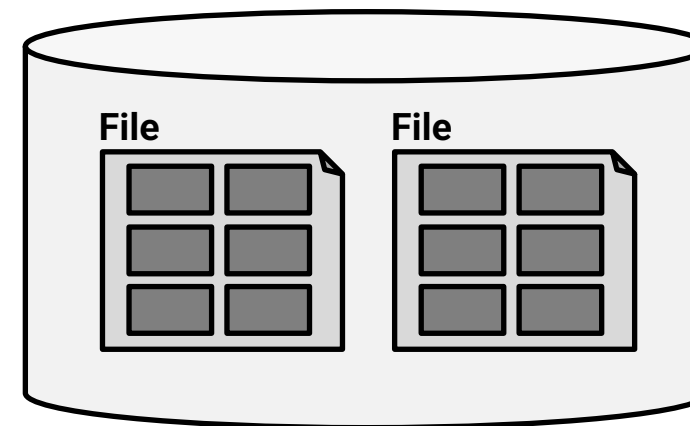
Conceptos claves

Archivos de bases de datos

- **Archivo de un DB:** Una colección de páginas, cada una conteniendo una colección de registros.

Table

ID	Last Name	First Name	Age	Salary
123	Simpson	Homer	31	\$400
443	Simpson	Marge	32	\$140
244	Flanders	Ned	55	\$300
134	Skinner	Seymour	55	\$400

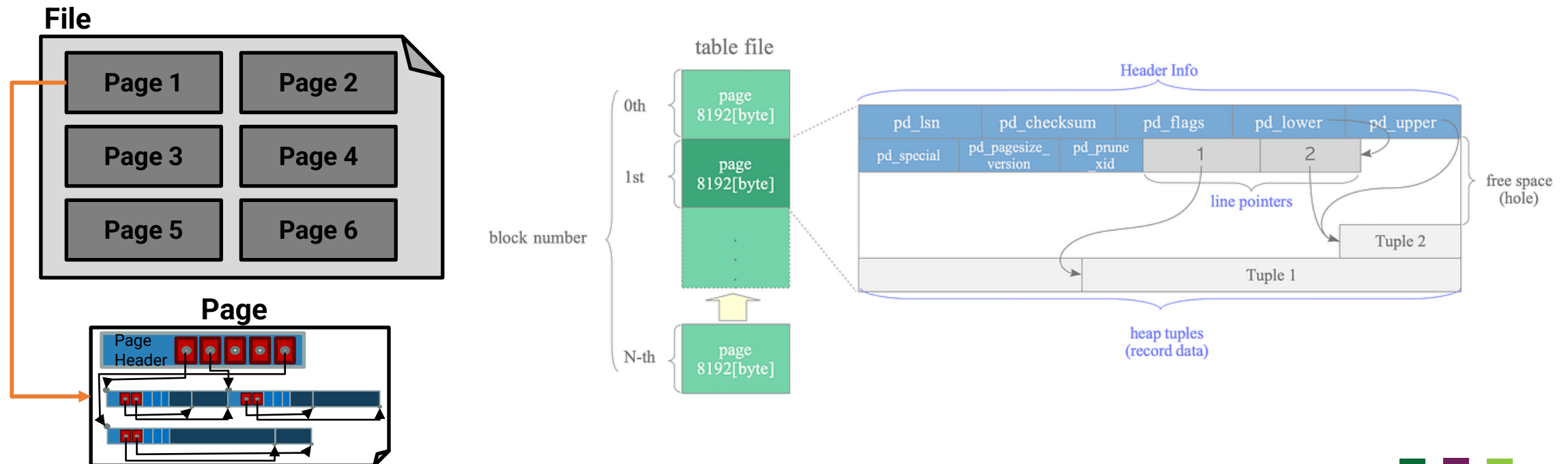


- **API para los niveles superiores del DBMS:**
 - Insertar/eliminar/modificar registro
 - Obtener un registro particular por **recordID** ...
 - El id de registro es un puntero que codifica el par (**pageID, ubicación en la página**)
 - Escanear todos los registros
 - Posiblemente con algunas condiciones sobre los registros a recuperar
- Puede abarcar múltiples archivos del SO e incluso máquinas
 - 0 dispositivos de disco “en bruto” (*raw*)

Conceptos claves

Paginas (1)

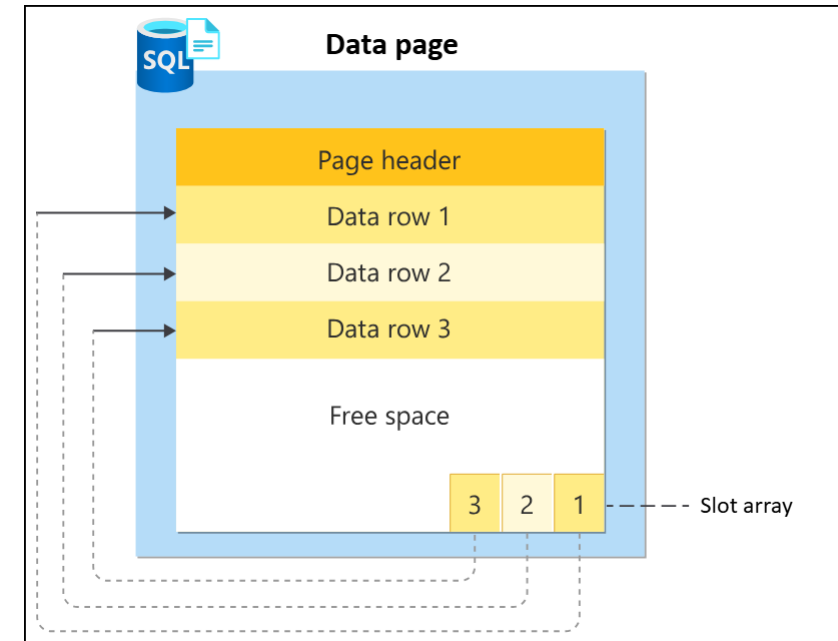
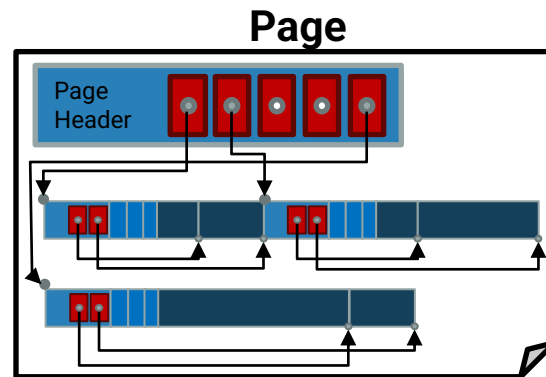
- Una página es un bloque de datos de tamaño fijo.
 - Puede contener **tuplas, metadatos, índices, registros de log...**
 - La mayoría de los sistemas no mezclan tipos de páginas.
- A cada página se le asigna un identificador único.



Conceptos claves

Paginas (2)

- Existen tres nociones diferentes de “páginas” en un DBMS:
 - Página de hardware (usualmente 4KB)
 - Página del SO (4KB)
 - Página de base de datos (1–16KB)
 - **SQLite:** 1 KB
 - **IMB DB2, Oracle:** 4 KB
 - **SQL Server, PostgreSQL:** 8 KB
 - **MySQL:** 16 KB



Por página de hardware, nos referimos al nivel en el cual el dispositivo puede garantizar una “escritura a prueba de fallos” (*failsafe write*).

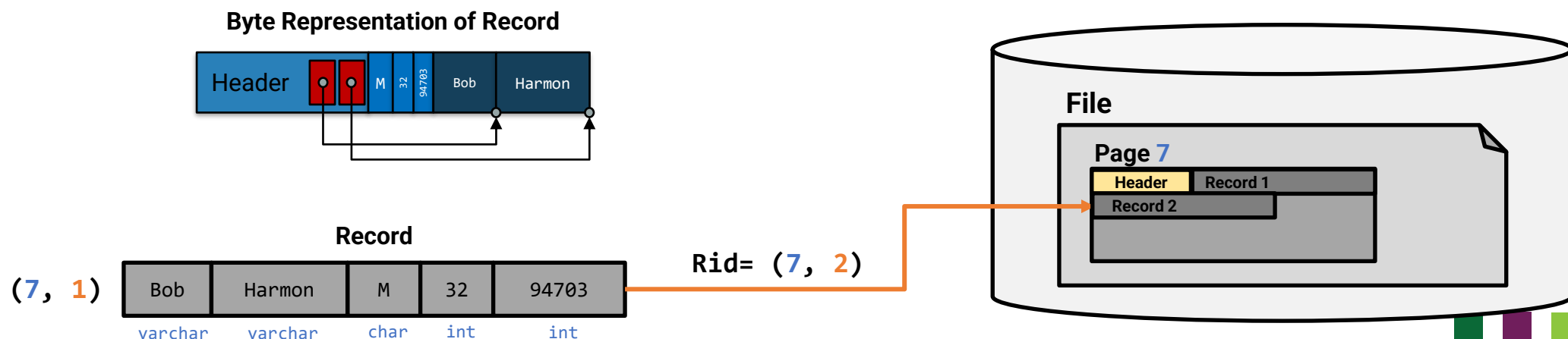
Conceptos claves

Registros - Record ID (RID)

- El DBMS necesita una forma de llevar el seguimiento de las tuplas individuales (**Registros**).
- A cada tupla se le asigna un identificador único de registro.
 - Lo más común:

$$\text{RID} = (\text{PageID}, \text{SlotNumber})$$

- También puede contener información sobre la ubicación en el archivo.
- El Record id (**RID**) es un identificador interno del DBMS (**metadata**), por lo tanto no tiene sentido lógico para una aplicación.



Estructura de los archivos bases de datos

Fundamentos de la organización de archivos en DB

- **Naturaleza Estructural:**
 - A diferencia de los archivos convencionales (flujos de bits), los archivos de bases de datos poseen una organización física definida.
 - El sistema gestiona los datos como una colección de registros ubicados en páginas o bloques de disco.
- **Factores que determina la organización:**
 - **Disposición Física:** Define cómo se posicionan los registros dentro de las páginas para optimizar el espacio.
 - **Eficiencia Operativa:** Determina el rendimiento (costo de E/S) en las tareas de lectura y escritura.
 - **Métodos de Acceso:** Establece la lógica necesaria para recuperar la información de manera ágil.
- **Objetivo Principal:**
 - Minimizar el movimiento de los cabezales del disco y reducir el tiempo de respuesta en las consultas.



Tipos de archivos de bases de datos

Clasificación

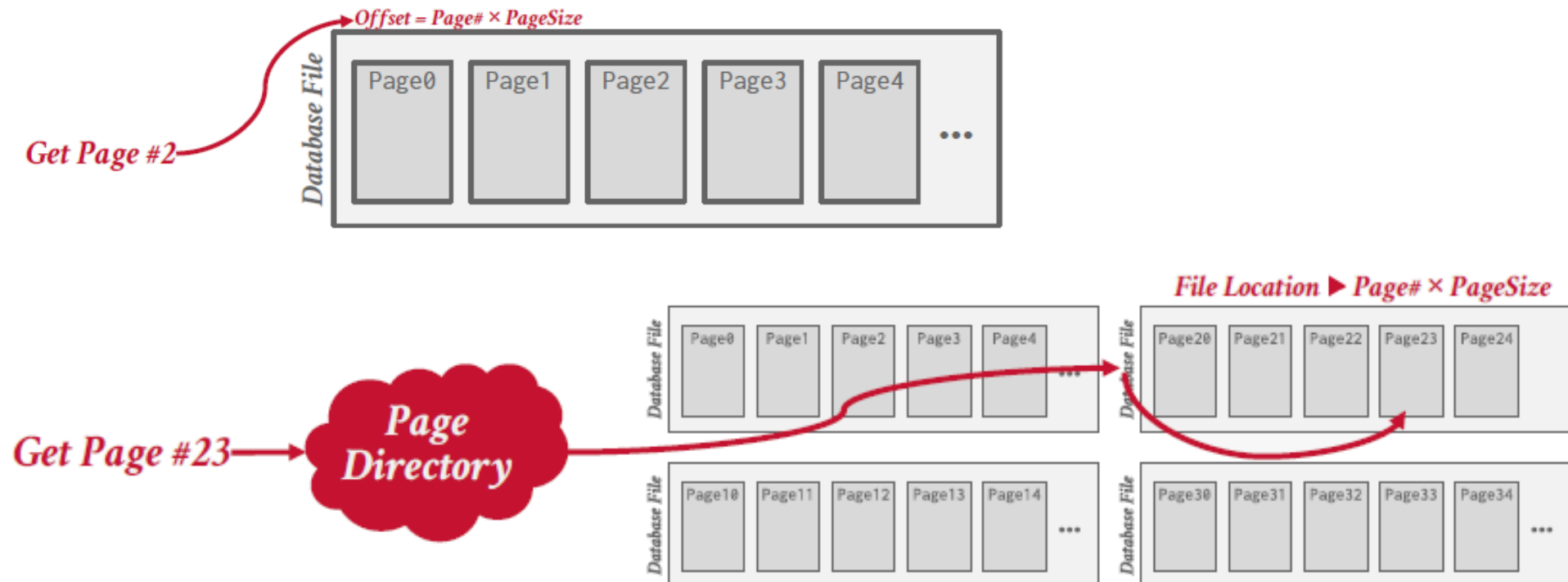
A continuación los diferentes tipos de archivos de bases de datos:

- **Archivos Heap no ordenados (Unordered Heap Files)**
 - Los registros se almacenan en orden aleatorio o en el orden en que fueron escritos.
 - Para localizar un registro específico, se requiere un escaneo completo o el uso de índices auxiliares
- **Archivos ordenados (Sorted Files)**
 - Los registros se mantienen en un orden físico basado en una clave de búsqueda.
 - Facilita las consultas de rango y la búsqueda binaria.
 - Las inserciones son costosas
- **Archivos Heap agrupados (Clustered Heap Files)**
 - Los registros y las páginas están agrupados
- **Archivos de índices (Index Files)**
 - Árboles B+, Hashing lineal, ...
 - Pueden contener registros o apuntar a registros en otros archivos



Unordered heap files

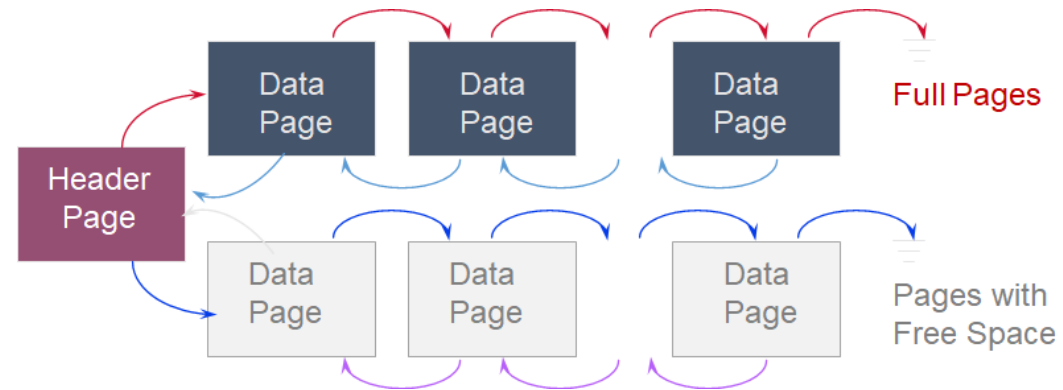
- Un **heap file** es una colección no ordenada de páginas con tuplas que se almacenan en orden aleatorio.
 - Crear / Obtener / Escribir / Eliminar Página.
 - También debe soportar la iteración sobre todas las página.
- Hace uso de **metadatos** adicionales para rastrear la ubicación de los archivos y la disponibilidad de espacio libre.



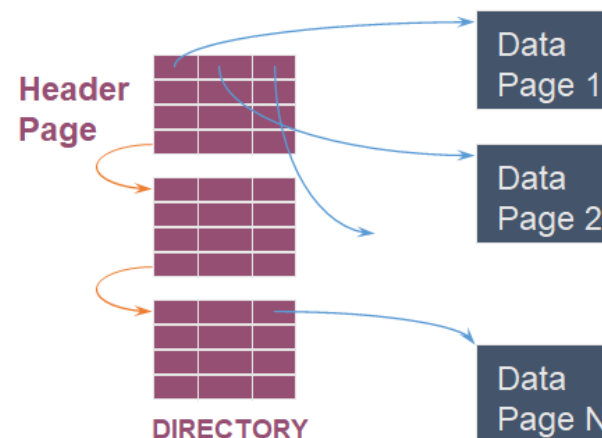
Unordered heap files

De acuerdo a como se organizan las paginas, existen dos alternativas principales para la implementación de **heap files**:

- Implementación como lista enlazada de paginas

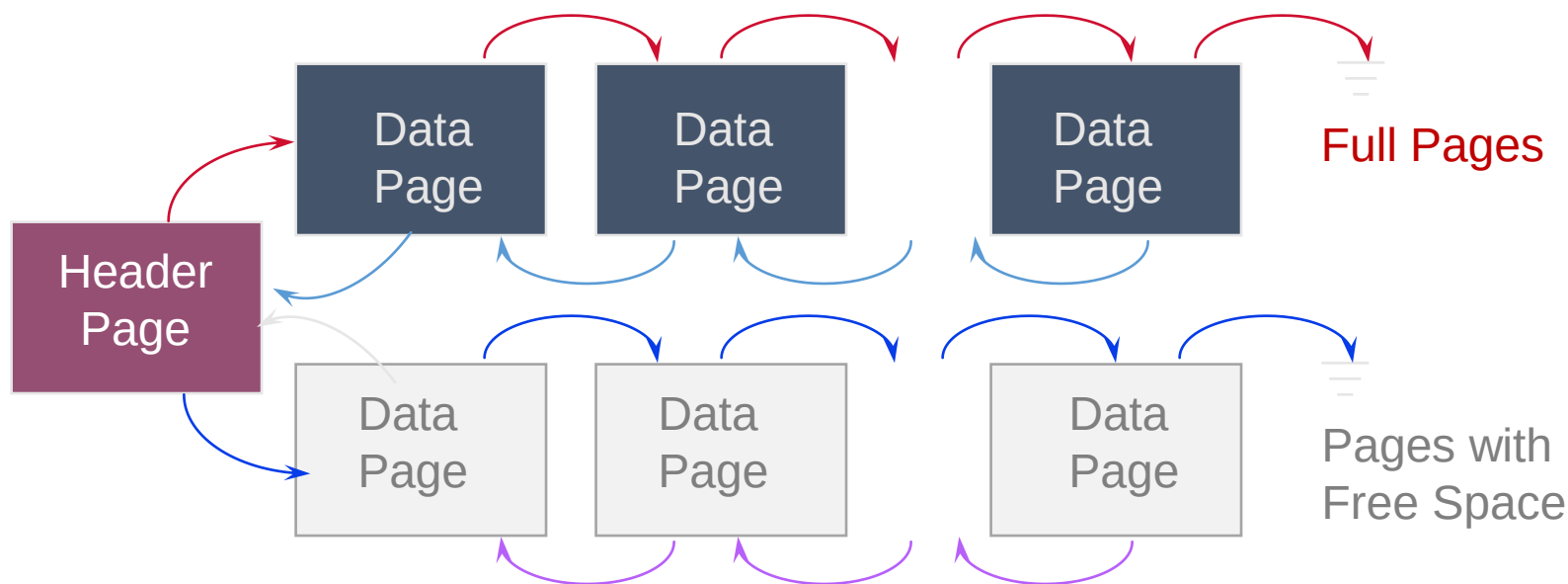


- Implementación como directorio de paginas



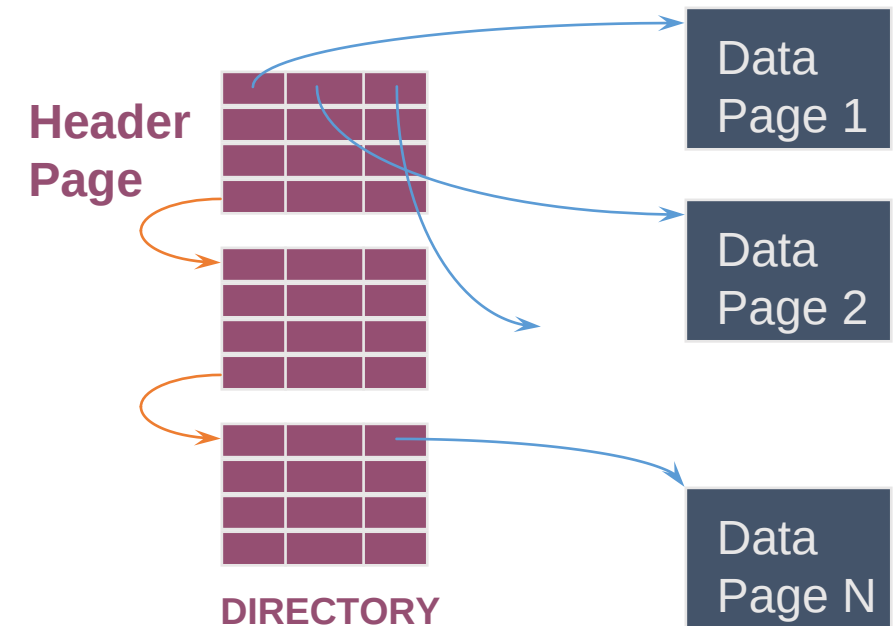
Unordered heap files – Lista enlazada

- El DBMS mantiene una página de encabezado (**header page**) que actúa como inicio del archivo y separa las paginas de datos en completas y libres:
 - **Lista de páginas llenas (Full Pages)**: Paginas que han agotado su capacidad total.
 - **Lista de páginas con espacio libre (Pages With Free Space)**: Paginas que aun pueden recibir registros.
- El ID de la página de cabecera (**header page id**) y el nombre del Heap file deben almacenarse en algún lugar del disco pues será el punto de partida para navegar por el resto del archivo.
- Cada página de datos contiene registros, un rastreador de espacio libre y punteros (desplazamientos de bytes) a la página siguiente y anterior.



Unordered heap files – Directorio

- La implementación del Directorio de Páginas se diferencia de la implementación de la Lista Enlazada en que solo utiliza una lista enlazada para las páginas de encabezado.
- Cada página de encabezado contiene un puntero (desplazamiento de bytes) a la siguiente página de encabezado, y sus entradas contienen tanto un puntero a una página de datos como el espacio libre disponible en dicha página.
- El ID de la página de cabecera (Header Page) inicial y el nombre del archivo se almacenan en el Catálogo de la Base de Datos para su acceso rápido.



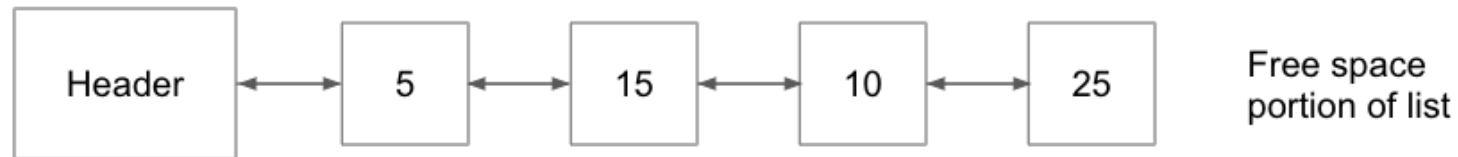
Comparación de las implementaciones

- La principal ventaja de los Directorios de Páginas sobre las Listas Enlazadas es que la inserción de registros suele ser más rápida.
 - Caso lista enlazada: Para encontrar una página con suficiente espacio, puede ser necesario leer la página de encabezado y cada página de la porción libre.
 - Caso implementación de Directorio: Solo requiere leer, como máximo, todas las páginas de encabezado, ya que contienen información sobre el espacio disponible en cada página de datos del archivo.
- **Ejemplo:** Un archivo heap se implementa como una Lista Enlazada y un Directorio de Páginas. Cada página ocupa 30 bytes y se inserta un registro de 20 bytes en el archivo. ¿Cuál sería el costo de acceso en términos de operaciones I/O?

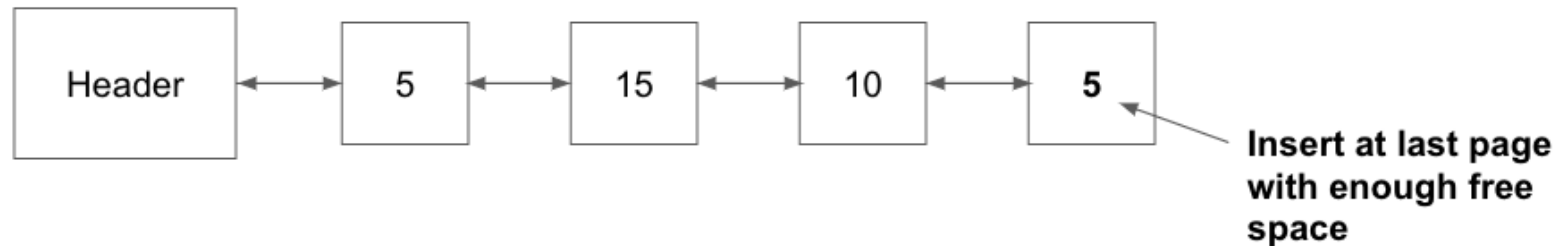
Comparación de las implementaciones

- Implementación como lista:

within box = # of free bytes



Insert 20 byte record

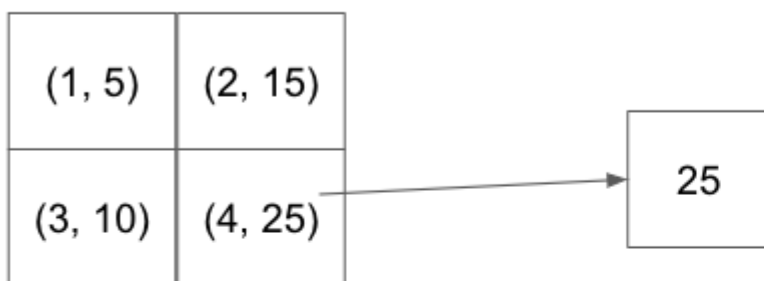


I/O Cost: 5 (read all pages) + 1 (write last data page) = **6 I/Os**

Comparación de las implementaciones

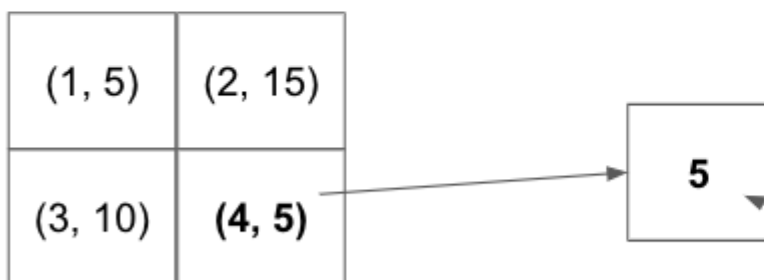
- Implementación como directorio:

(#, #) within box = (page #, # of free bytes)



Header Page

Insert 20 byte record



Header Page

Insert at last page
with enough free
space

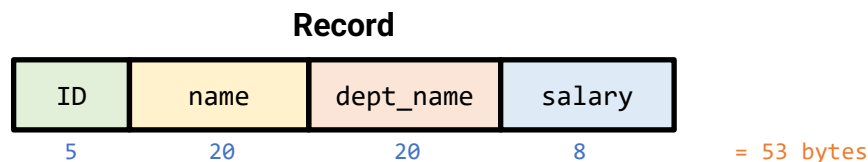
I/O Cost: 1 (read header) + 1 (read data) + 1 (write data) + 1 (write header) = 4 I/Os

Definición del problema

- Suponga que se tiene una base de datos universitaria con un archivo de registros tipo **instructor** cuyos campos (**fields**) se definen a continuación.

```
type instructor = record
  ID : varchar (5);
  name : varchar(20);
  dept_name : varchar(20);
  salary : numeric (8,2);
end;
```

- Suponga que cada carácter ocupa 1 byte y que numeric (8,2) ocupa 8 bytes.

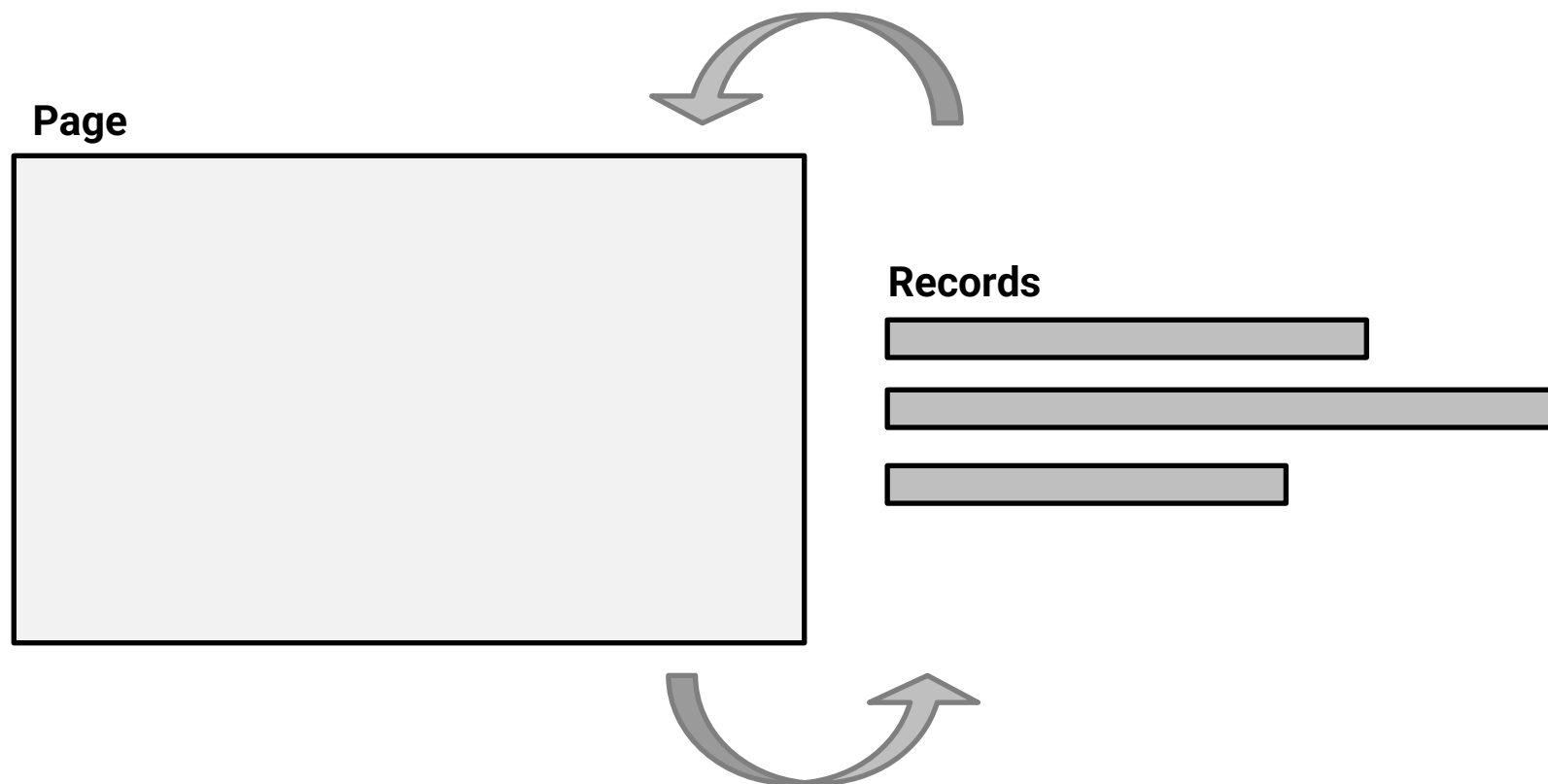


<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

Definición del problema

- **Pregunta:** ¿Cómo almacenar registros en una página/archivo? de tal forma que:
 - Se pueda apuntar a cada uno de los registros
 - Sea posible insertar/eliminar registros empleando pocos accesos al disco

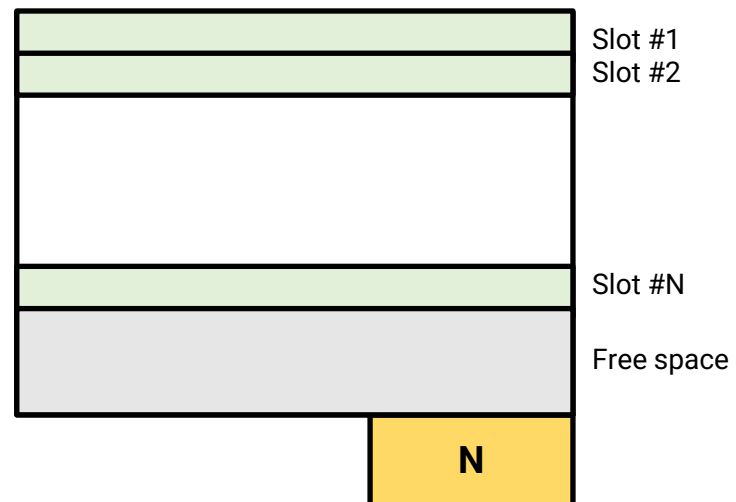


Tipos de registros

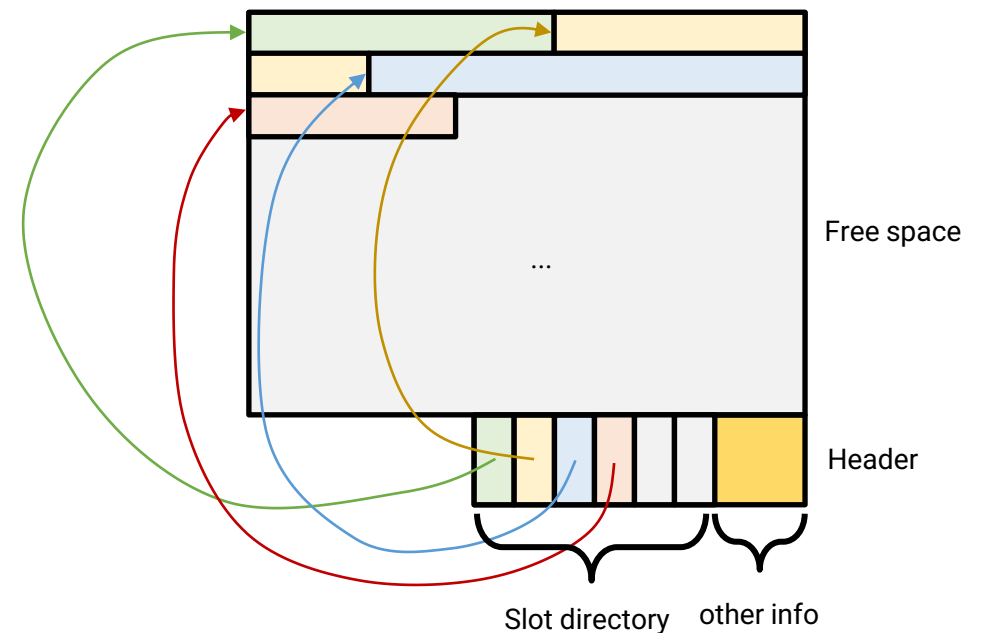
Longitud fija

Los tipos de registros están completamente determinados por el esquema de la relación y se presentan en 2 tipos:

Registros de Longitud Fija (FLR - Fixed Length Records)



Registros de Longitud Variable (VLR - Variable Length Records)



Registros de longitud fija

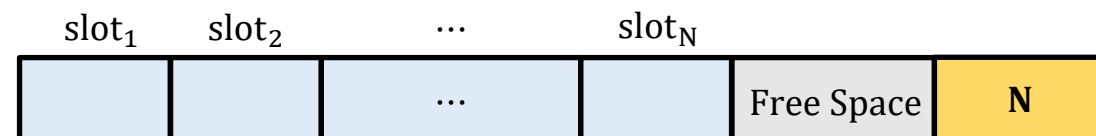
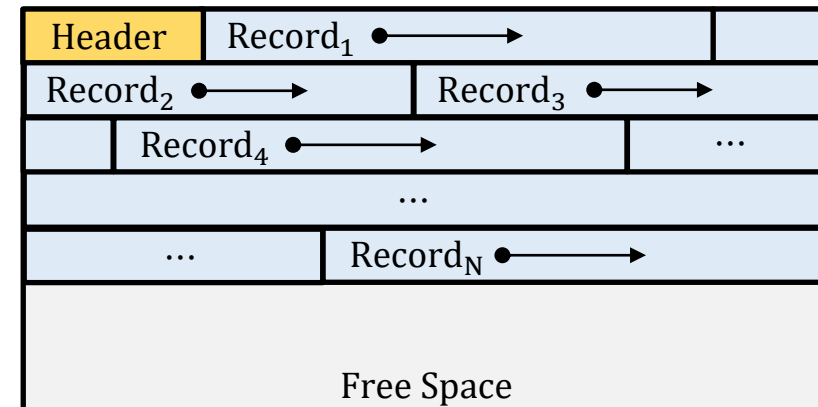
- En este formato, cada registro de una tabla tiene exactamente el mismo tamaño.
- Cada uno de los registros del archivo tienen un tipo particular de dato.
- Diferentes archivos son empleados para diferentes relaciones.
- Este es el caso mas fácil de implementar.
- **Suposición:** Se asumirá que los registros son mas pequeños que una pagina.

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Registros de longitud fija

- La pagina se divide en **slots**.
- Cada slot tiene una tupla (registro).
- El RecordID (rid) para cada tupla es (PageId, slotNum)

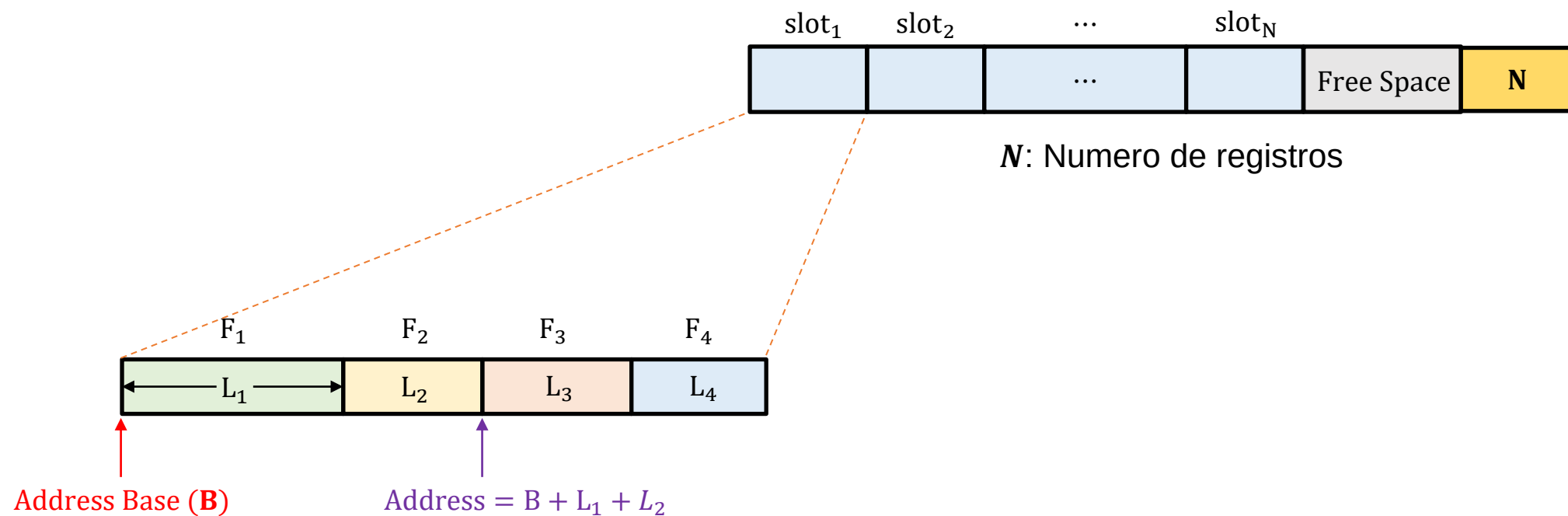
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000



N: Numero de registros

Registros de longitud fija

- La información sobre los tipos de campos es la misma para todos los registros en un archivo; se almacena en los **catálogos del sistema**.
- Encontrar el campo i -ésimo se realiza mediante aritmética.



F_i : Campo i

L_i : Longitud campo i

Registros de longitud fija - Inserción

- Almacenar el registro i comenzando desde el byte $n \times (i - 1)$, donde n es el tamaño de cada registro.
- El acceso al registro es simple:

$$Address(Rec_i) = Address(page) + (Address(slot) \times Size(Rec_i))$$

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

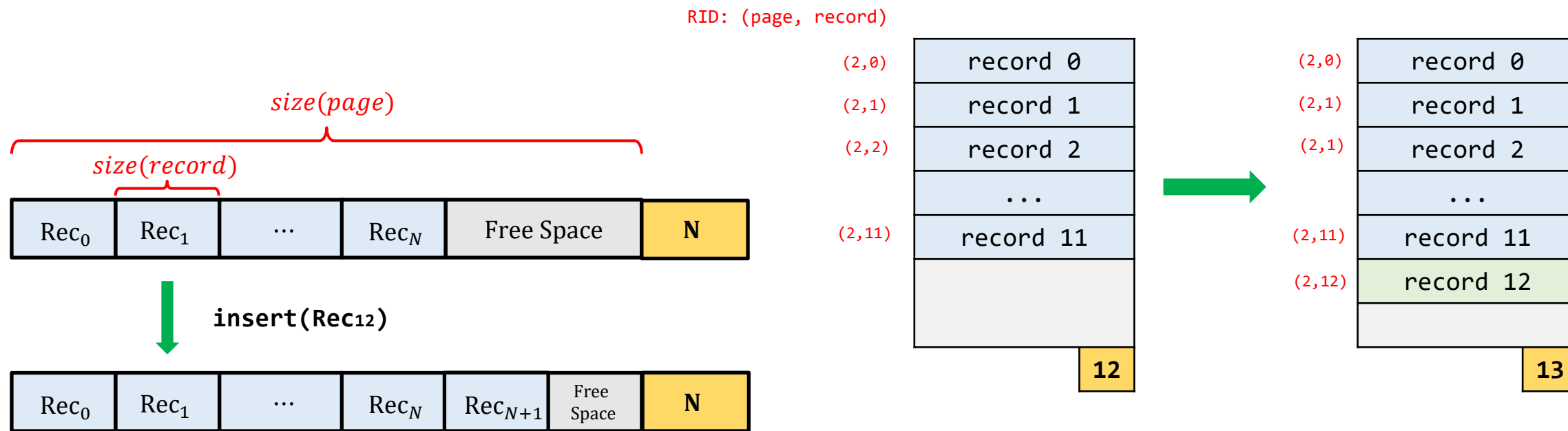
record 0
record 1
record 2
...
record 11
12

Metadatos



Registros de longitud fija - Inserción

• Inserción:



- **Problema:** Un registro de la misma tabla cruzar el límite de página.
 - Si $(\text{size}(\text{Page})) \% \text{size}(\text{record}) \neq 0$, un registro quedará entre dos paginas diferentes.
 - Se necesitan dos accesos para leer el registro.

Registros de longitud fija - Eliminación

¿Que hacer cuando se elimina el registro i ?

Alternativas:

1. Mover los registros $i+1, \dots, n$ a $i, \dots, n-1$
2. Mover el registro n a i
3. No mover los registros, solo linkear todos los registros libres a los registros **Free list**.

Borrar el Registro 3 (**record 3**)

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Registros de longitud fija - Eliminación

Alternativa 1 – Mover registros:

- Cuando se borra un registro se puede desplazar el registro situado a continuación al espacio ocupado anteriormente por el registro borrado y hacer lo mismo con los demás registros hasta que todos los registros situados a continuación del borrado se hayan desplazado hacia delante.

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

remove(record 3)

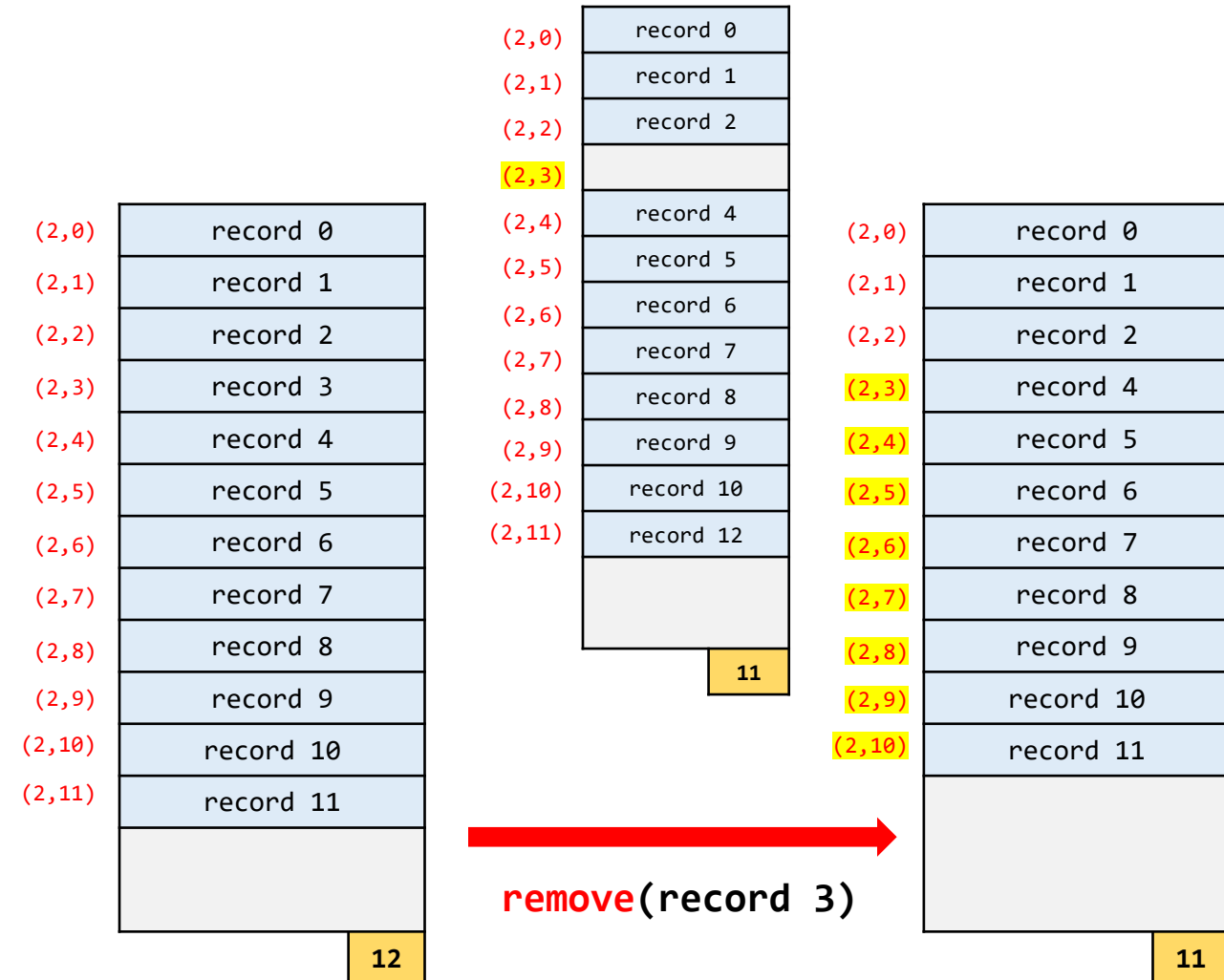


record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Registros de longitud fija - Eliminación

Problema:

- La reorganización de la página al mover varios archivos, para lograr la compactación, es costosa.
- Los apuntadores a los RID (Register ID) deben ser actualizados.



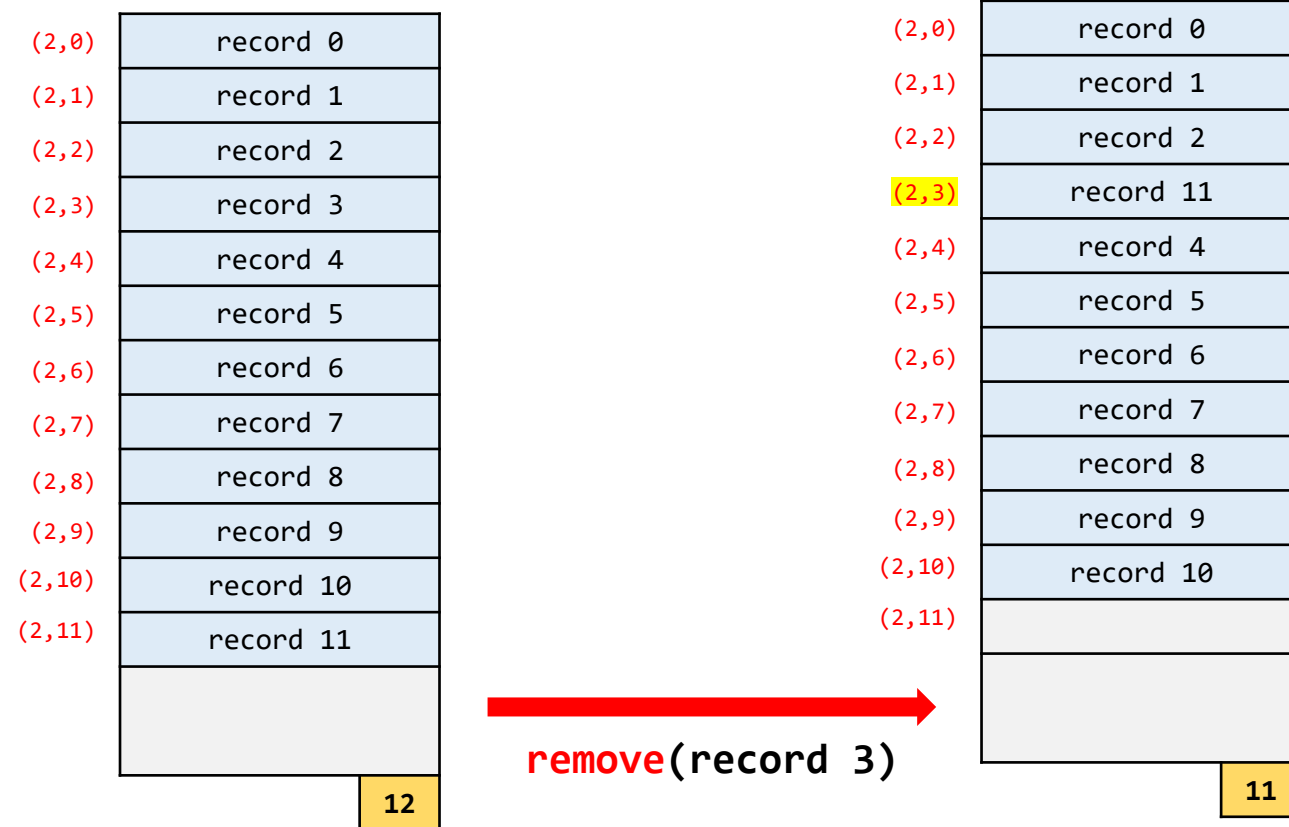
Registros de longitud fija - Eliminación

Alternativa 2 – Mover el ultimo registro:

- Este caso es mas simple, pues lo que se hace es desplazar simplemente el último registro del archivo al espacio ocupado por el registro borrado

Alternativa:

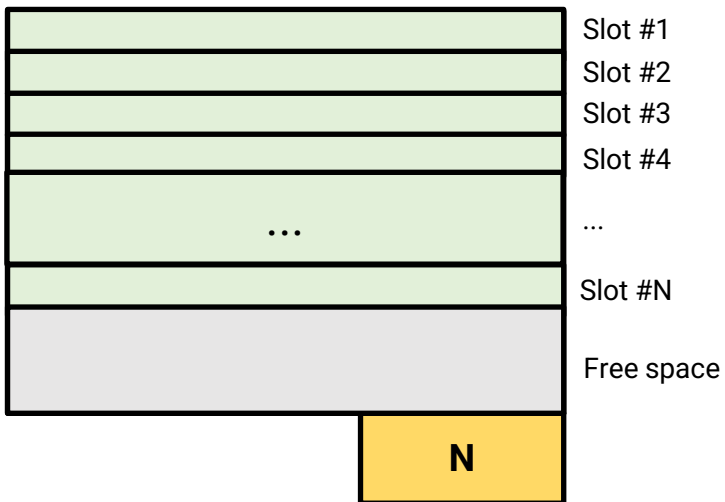
- Dejar libre el espacio ocupado por los registros borrados y esperar a una inserción posterior antes de volver a utilizar ese espacio.
- Es necesario introducir una estructura adicional en el header.**



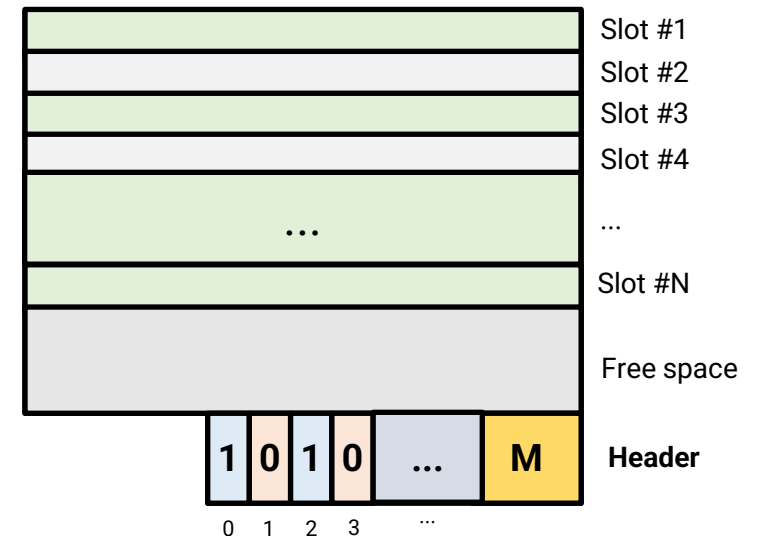
Registros de longitud fija – Uso de bitmap

Actualizando la estructura:

Registros de Longitud Fija (FLR - Fixed Length Records)



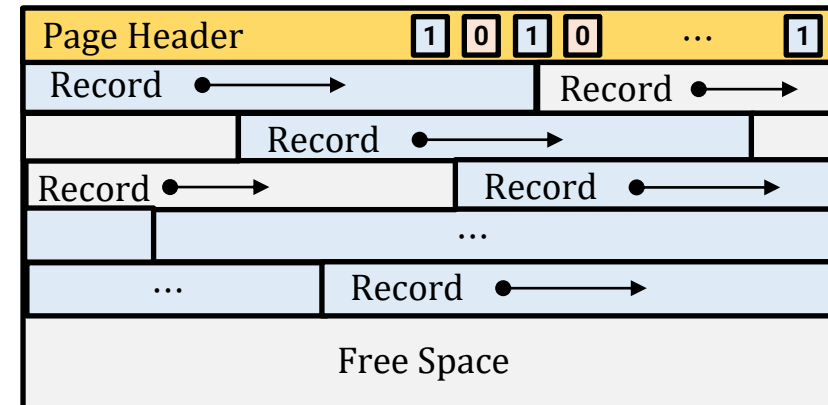
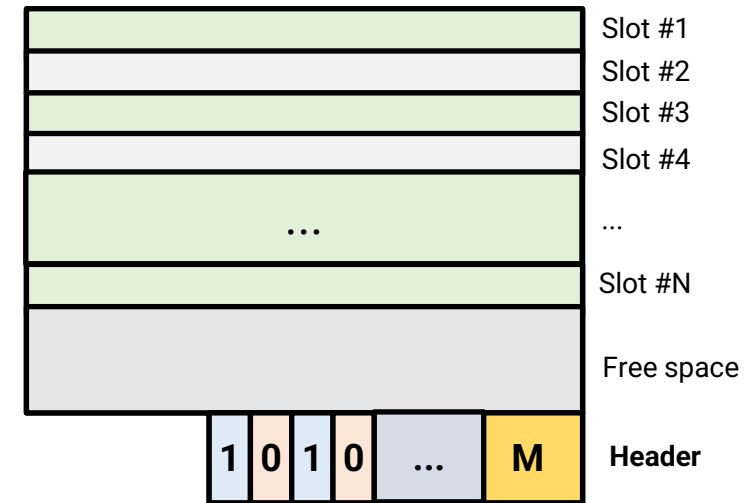
Registros de Longitud Fija (FLR - Fixed Length Records) – Con mapa de bits



Registros de longitud fija – Uso de bitmap

Enfoque Desempaquetado con Mapa de Bits (Bitmap)

- Esta técnica utiliza una estructura auxiliar, generalmente un **array de bits bitmap** ubicado en la cabecera de la página, donde cada bit corresponde a una slot (ranura) específica.
- La indicación del estado de los slots (ranuras) se hace en el bitmap:
 - 1: Significa que hay un registro en el slot
 - 0: El slot esta vacío
- Ventajas:**
 - Los registros no necesitan moverse físicamente para llenar huecos.
 - Los identificadores de registro (RID) permanezcan estables (nos es necesario actualizarlos).



Registros de longitud fija

Inserción:

- Encontrar el primer bit del **array map** en 0 (slot vacío), llenar el slot con el registro y actualizar el bit a 1 (slot ocupado)

(2,0)	record 0														
(2,1)	record 1														
(2,2)	record 2														
(2,3)	record 3														
(2,4)	record 4														
(2,5)	record 5														
(2,6)	record 6														
(2,7)	record 7														
(2,8)	record 8														
(2,9)	record 9														
(2,10)	record 10														
(2,11)	record 11														
	1	1	1	1	1	1	1	1	1	1	1	1	0	0	12

insert(record 12)



(2,0)	record 0															
(2,1)	record 1															
(2,2)	record 2															
(2,3)	record 3															
(2,4)	record 4															
(2,5)	record 5															
(2,6)	record 6															
(2,7)	record 7															
(2,8)	record 8															
(2,9)	record 9															
(2,10)	record 10															
(2,11)	record 11															
(2,12)	record 12															
<table><tr><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td><td>13</td></tr></table>		1	1	1	1	1	1	1	1	1	1	1	1	1	0	13
1	1	1	1	1	1	1	1	1	1	1	1	1	0	13		

Registros de longitud fija – Uso de bitmap

Eliminación:

- Borrar el **bit** (ponerlo a 0) del **bitarray** correspondiente al registro que se elimina.

(2,0)	record 0
(2,1)	record 1
(2,2)	record 2
(2,3)	record 3
(2,4)	record 4
(2,5)	record 5
(2,6)	record 6
(2,7)	record 7
(2,8)	record 8
(2,9)	record 9
(2,10)	record 10
(2,11)	record 11
1 1 1 1 1 1 1 1 1 1 1 1 0 0 12	

remove(record 3)

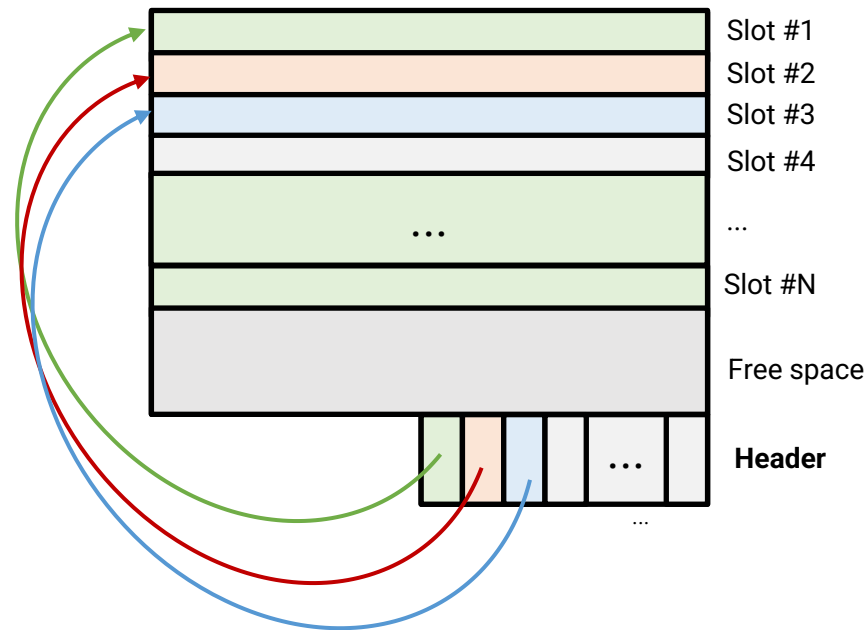


(2,0)	record 0
(2,1)	record 1
(2,2)	record 2
(2,3)	record 3
(2,4)	record 4
(2,5)	record 5
(2,6)	record 6
(2,7)	record 7
(2,8)	record 8
(2,9)	record 9
(2,10)	record 10
(2,11)	record 11
1 1 1 0 1 1 1 1 1 1 1 1 0 0 11	

Registros de longitud fija– Slotted page

Desventajas:

- Es el empleado en la actualidad.
- Se emplea para paginas con registros de diferente tamaño.
 - Se profundizara luego



Referencias

- <https://www.mongodb.com/resources/basics/databases/nosql-explained>
- <https://blog.bytebytego.com/p/sql-vs-nosql-choosing-the-right-database>
- <https://cs186berkeley.net/notes/note17/>
- <https://levelup.gitconnected.com/postgresql-deep-dive-the-anatomy-of-disk-file-and-page-layout-part-2-9b6fb7ede383>
- <https://medium.com/@pablo.lopez.santori/what-do-postgres-tables-actually-look-like-426028eabc1a>
- <https://michalpittr.substack.com/p/how-does-sqlite-store-data>
- <https://cs186berkeley.net/notes/note3/>
- <https://github.com/MIT-DB-Class/simple-db-hw>
- <https://simplifiedb-java.netlify.app/#/>
- <https://github.com/quazi-irfan/pySimpleDB>
- <https://tinydb.readthedocs.io/en/latest/>

UNIVERSIDAD DE ANTIOQUIA

Curso de Estructuras de datos
Clase 4 – Archivos (Parte 1)